

Harness Engineering

Super AI Engineer Season 6

Nutchanon Yongsatianchot
3CHA.AI



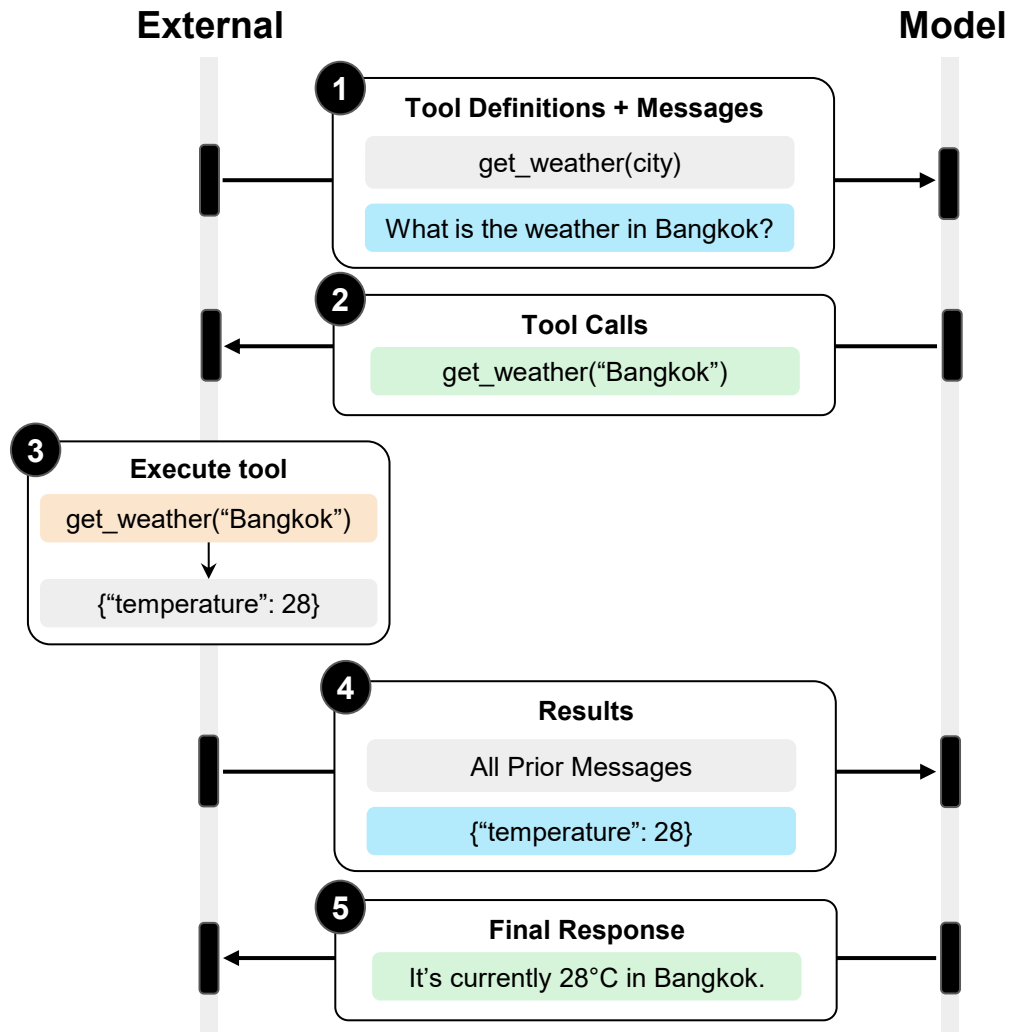
Q: Do you still have OpenAI API left?

Recap

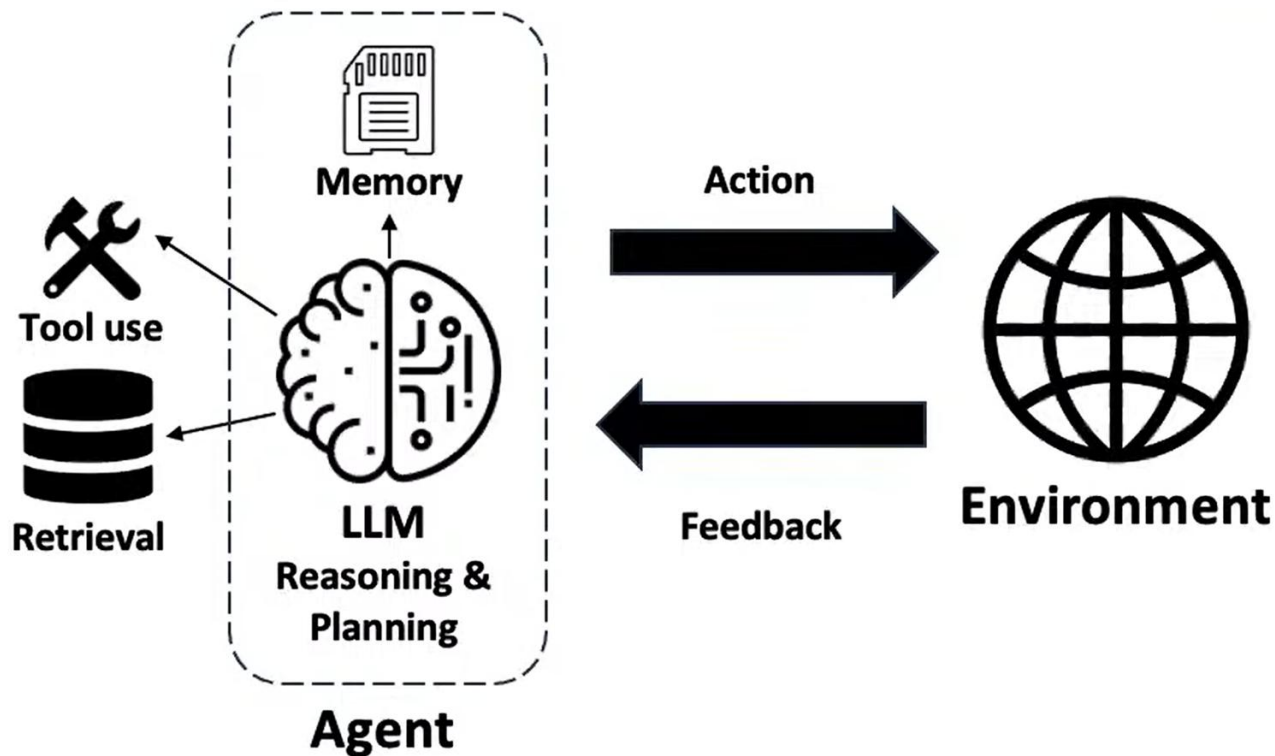
- **Recap**
- Harness Engineer
- Context Management
- Caching
- Multi-agent patterns
- Harness Examples

Recap: Tool Use

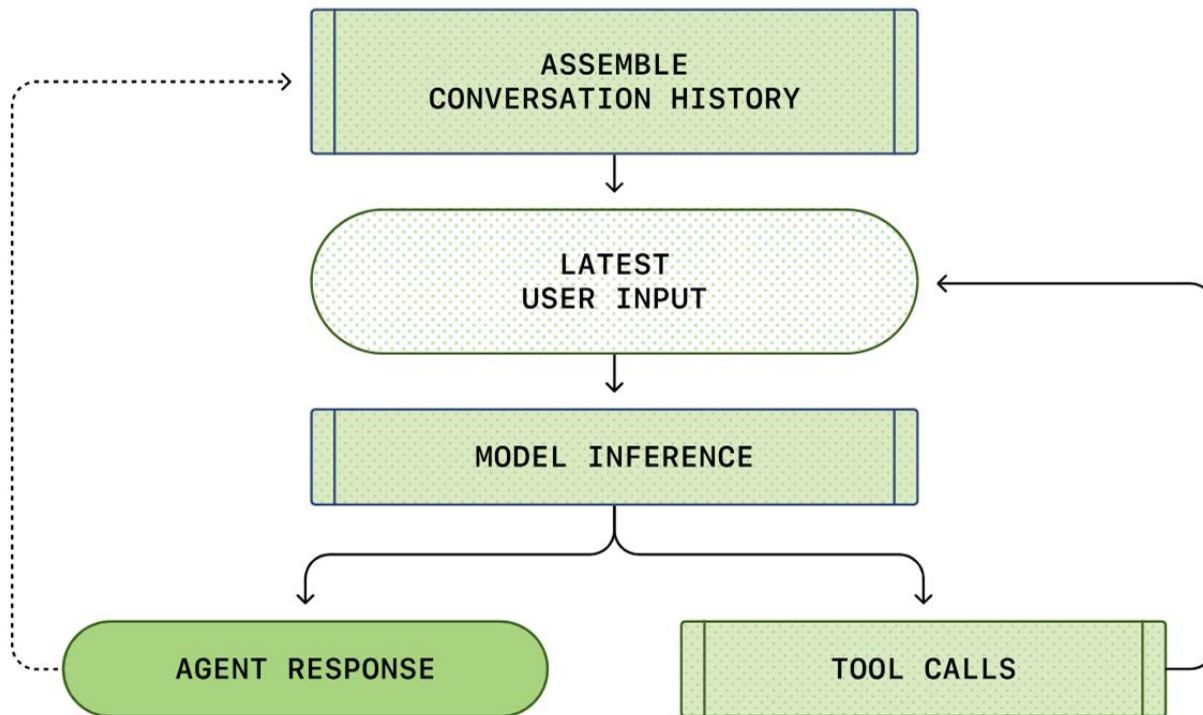
- Provide tools to LLMs and they can decide what tools to use and when.
- A tool (function) is called separately and the outputs are then appended back to the conversation.



Recap: Agent = “LLM in a loop” (ReAct loop)



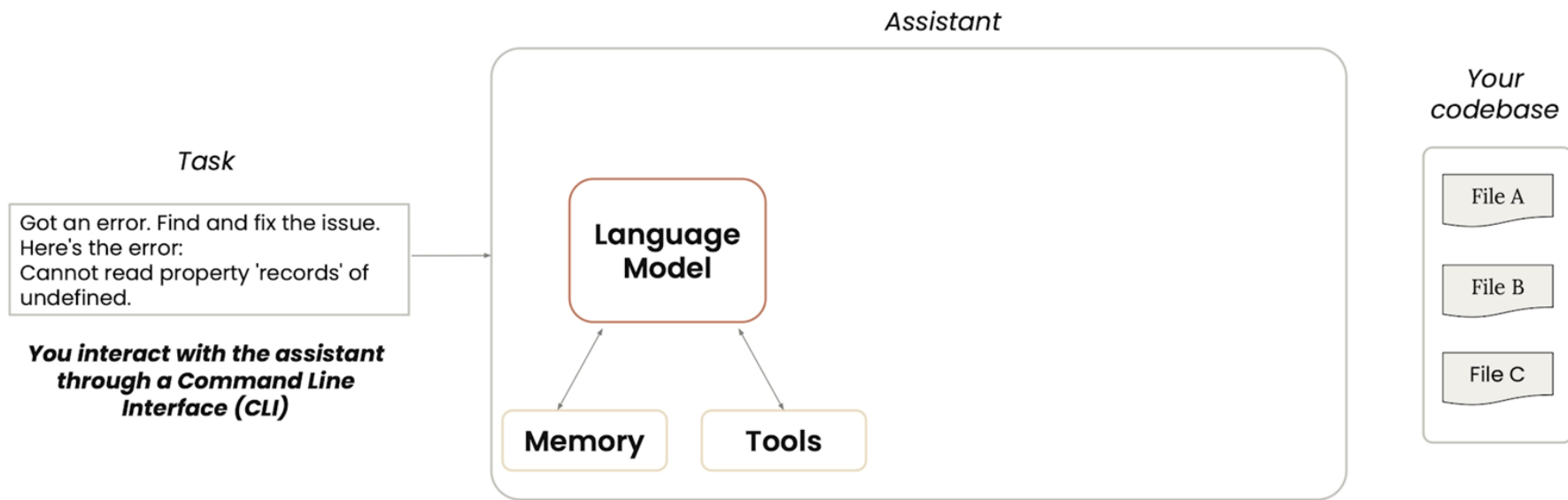
Multi-turn agent loop



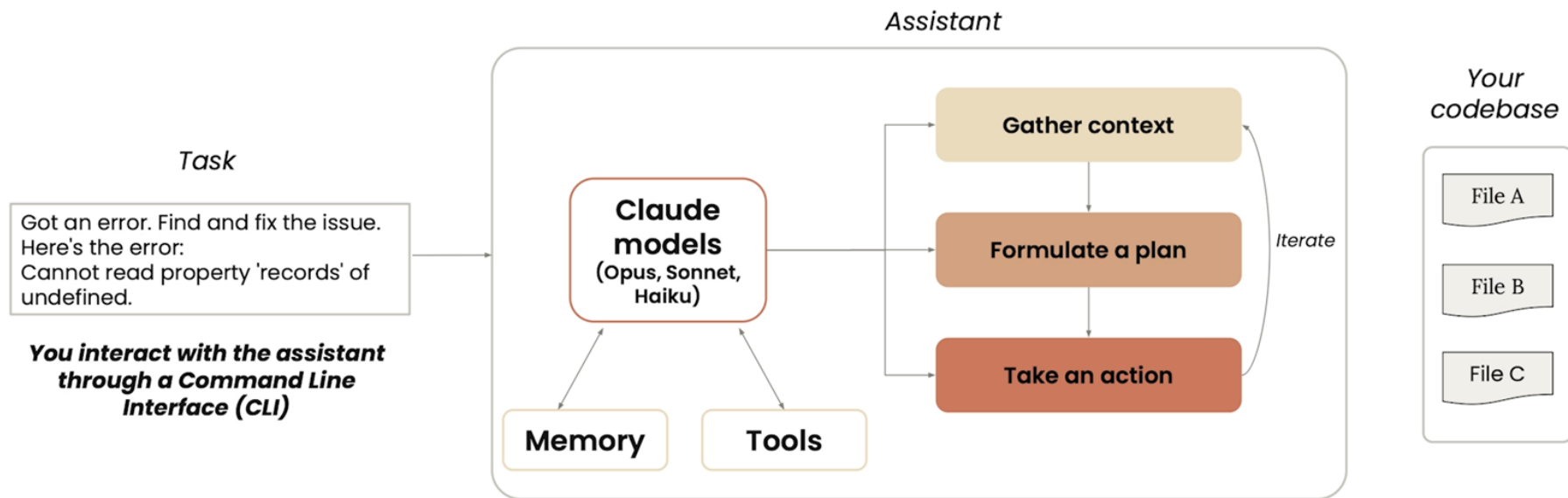
CLI Agentic Coding Assistant



CLI Agentic Coding Assistant



CLI Agentic Coding Assistant



Harness Engineering

- Recap
- **Harness Engineer**
- Context Management
- Caching
- Multi-agent patterns
- Harness Examples



Addy Osmani ✓
@addyosmani



Agent Harness Engineering

Agent Harness Engineering

83

566

3.3K

834K



Agent Harness Engineering: A Survey

Junjie Li^{1,6,*}, Xi Xiao^{6,*}, Yunbei Zhang^{5,*}, Chen Liu^{2,*}, Lin Zhao⁴, Xiaoying Liao³, Yingrui Ji⁶, Janet Wang⁵, Jianyang Gu⁷, Yingqiang Ge⁹, Weijie Xu⁹, Xi Fang⁹, Xiang Xu⁹, Tianchen Zhao⁹, Youngeun Kim⁹, Tianyang Wang⁶, Jihun Hamm⁵, Smita Krishnaswamy², Jun Huan^{9,†}, Chandan K Reddy^{8,9,†}

¹Carnegie Mellon University · ²Yale University · ³Johns Hopkins University · ⁴Northeastern University ·

⁵Tulane University · ⁶University of Alabama at Birmingham · ⁷The Ohio State University · ⁸Virginia Tech · ⁹Amazon

*Equal contribution. †Corresponding authors.

Paper

OpenReview

Huggingface

GitHub

Website

BibTeX

Welcome to Learn Harness Engineering

Learn Harness Engineering is a course dedicated to the engineering of AI coding agents. We have deeply studied and synthesized the most advanced Harness Engineering theories and practices in the industry. Our core references include:

- OpenAI: Harness engineering: leveraging Codex in an agent-first world
- Anthropic: Effective harnesses for long-running agents
- Anthropic: Harness design for long-running application development
- Awesome Harness Engineering



TRAE ✓
@Trae_ai



Research Log

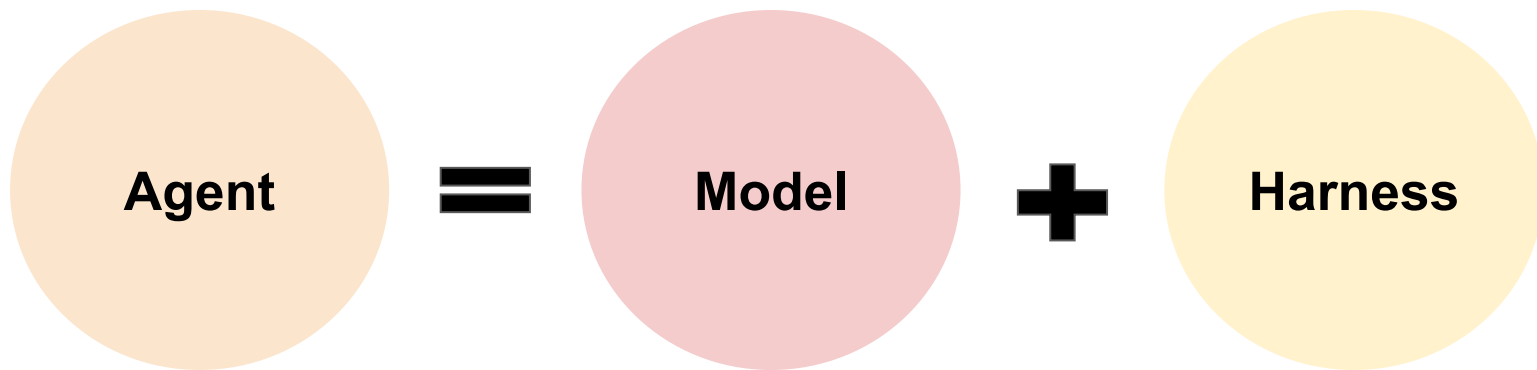
TRAE

The Definitive Guide To
Harness Engineering

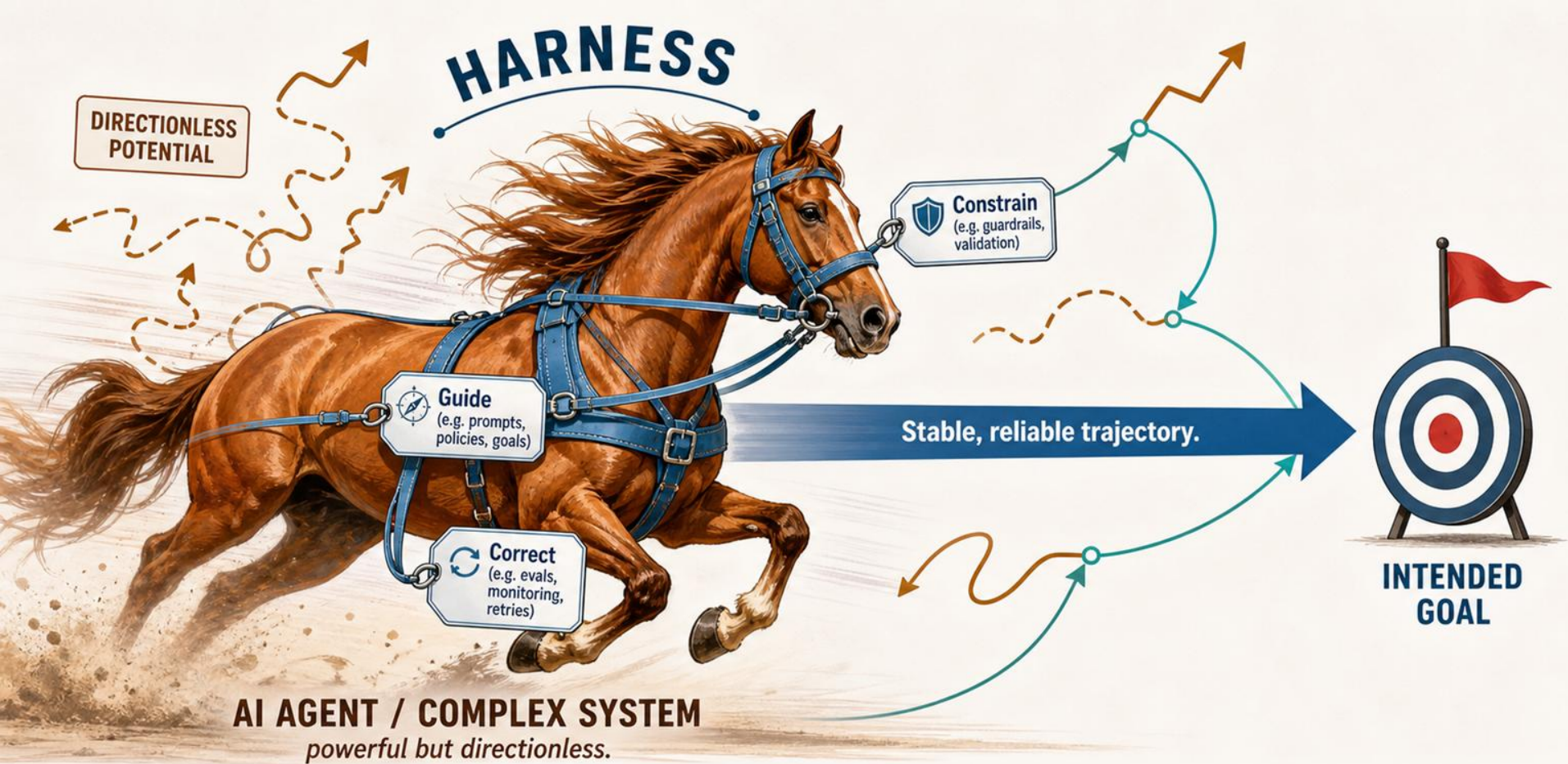


<https://x.com/addyosmani/status/2053231239721885918?s=12>

What is Harness?



If you're not the model, you're the harness.



The horse provides the **power**.

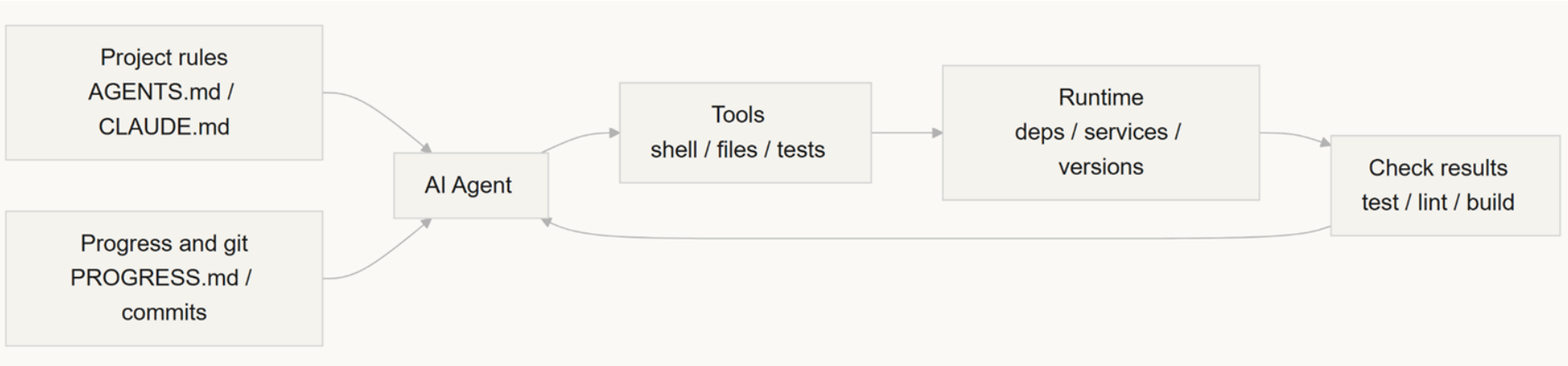


The harness provides the **direction**.

The Harness is essentially **every piece of infrastructure other than the LLM** that enables an agent to actually deliver results. It isn't about "better prompts" or "more capable models". **It's about optimizing the environment and mechanisms the model operates within.** It is an engineering philosophy and framework designed to transform raw AI intelligence into **reliable, controllable, and scalable productivity.**

Concretely, a harness includes:

- System prompts, AGENTS.md, skill files, and subagent instructions.
- Tools, skills, MCP servers, and their technical descriptions.
- Bundled infrastructure: filesystem, sandboxes, and headless browsers.
- Orchestration logic for spawning subagents, handling handoffs, and routing models.
- Hooks and middleware for deterministic execution, like lint checks or context compaction.
- Observability tools for logs, traces, cost, and latency metering.



Prompt Engineering

Single-call Optimization
Optimize the **model input**



Instruction & Role

*Define task, persona
and constraints*



Example & Format

*Define task, persona
and constraints*



Prompt Iteration

*Refine wording from
observed failures*

Context Engineering

Multi-step Context Optimization
Optimize **what the model sees**



Retrieval & Loading

*Bring in relevant files, docs
and tool outputs*



Memory & State

*Track progress, decisions,
and prior steps*



Compression & Filtering

*Keep useful context,
and remove stale information*

Harness Engineering

System-level Optimization
Optimize **how the model runs**



Tools & Environment

*Run action safely and
expose callable tools*



Orchestration

*Plan, act, verify, retry,
or delegate*



Verification & Evaluation

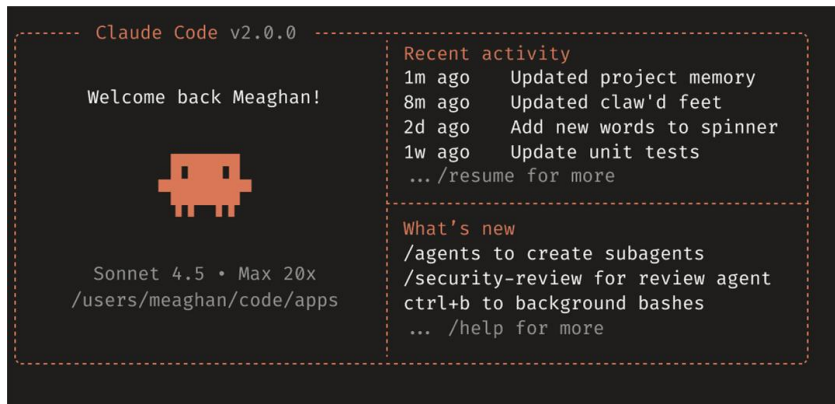
*Trace behavior, measure cost,
and check results*



Governance & Security

*Enforce permissions, approvals
and audit rules*

Examples of Harness



“At its core, an agent is a system that runs tools in a loop to achieve a goal.”

The real skill lies in designing both the tools and that loop.

The harness-engineering mindset

- The current frontier models are good enough.
- Failures are usually somewhat legible.
- If there are failures, it's likely not due to models.
 - If the agent ignored a convention, add it to [AGENTS.md](#).
 - If it ran a destructive command, write a hook to block it.
 - If it got lost in a 40-step task, split the architecture into a planner and an executor.

"It's not a model problem. It's a configuration problem." - HumanLayer

Why capable models still fail?

- **Vague requirements** — the agent can only guess. "Add a search feature" — that sentence means almost nothing.
- **Implicit conventions not written down** — the agent has no way to comply. It's not that it doesn't want to comply, it literally has never seen the rule.
- **Incomplete environment setup** — the agent spends energy fixing the environment. Incomplete dev setup, missing dependencies, wrong tool versions.
- **No verification methods** — the agent calls it done when it feels done. No tests, no lint, or verification commands that were never communicated to the agent.
- **Cross-session state loss** — every new session starts from scratch. All discoveries from the previous session are lost. Every new session has to re-explore the project structure and re-understand the code organization.

The gap between what today's models can theoretically do and what you actually see them doing is largely a *harness gap*.

The Ratchet: Every mistake becomes a rule

- **Agent mistakes = signals**
- **Add an instruction to avoid the mistakes.**
 - e.g., If an agent ships a PR with a commented-out test that gets merged by accident, that is an input. The next iteration of AGENTS.md must state: "Never comment out tests; delete or fix them."
- Constraints should only be added when you observe a real failure, and removed only when a capable model renders them redundant.

Working backwards from behavior

- To design a harness, start with the desired behavior and build the component that delivers it: Behavior we want → Harness design to achieve it.
- Every piece of the harness must have a distinct job. **If you cannot name the specific behavior a component exists to deliver, it should be removed.**



Every harness feature is a bridge across a specific thing the model can't do on its own.

Filesystem and Git - durable state

- **Filesystem:** Models can only operate on what fits in their context window. A filesystem provides a workspace to read data, a place to offload intermediate work, and a surface for multiple agents to coordinate.
- **Git** provides free versioning, allowing the agent to track progress, branch experiments, and rollback errors.

Bash and Code Execution: general-purpose tooling

- Most agents operate on a ReAct loop: reason, act via a tool call, observe, repeat. Instead of pre-building a tool for every conceivable action, giving the agent bash access allows it to build what it needs on the fly.
- Agents generally excel at shell commands, making bash and code execution the default strategy for autonomous problem-solving.

Sandboxes and Default Tooling

- Bash is only useful if it runs safely. Sandboxes provide agents with an isolated environment to run code, inspect files, and verify work without risking the host machine.
- A good sandbox ships with strong defaults: pre-installed language runtimes, test CLIs, and headless browsers, allowing the agent to observe its own work and close the self-verification loop.

Memory and Search: Continual Learning

- Models have no knowledge beyond their training weights and current context. Harnesses bridge this gap using memory files (like AGENTS.md) that inject knowledge into every session.
- For real-time information like new library versions or live data web search and MCP tools are baked directly into the harness.

Battling Context Rot

Models degrade in reasoning as their context windows fill up. Harnesses manage this scarcity using three primary techniques:

- **Compaction:** Intelligently summarizing and offloading older context to prevent API errors.
- **Tool-call offloading:** Storing massive tool outputs (like 2,000-line logs) in the filesystem while keeping only the essential headers and footers in context.
- **Progressive disclosure:** Revealing instructions and tools only when a task explicitly requires them, rather than loading everything at startup.

Long-Horizon Execution

Autonomous, long-running work suffers from early stopping and poor problem decomposition. Harnesses counter this through structural design:

- **Loops:** Intercepting a model's attempt to exit and forcing it to continue against a completion goal in a fresh context window.
- **Planning:** Forcing the model to decompose goals into a step-by-step plan file, checking its work via self-verification hooks after each step.
- **Splits:** Separating generation and evaluation into distinct agents, preventing the inherent positive bias models have when grading their own work.

Hooks are your Enforcement Layer

- **Hooks** (a script) bridge the gap between requesting an action and enforcing it. **They run at specific lifecycles: before a tool call, after a file edit, or before a commit.** Hooks block destructive commands, enforce auto-formatting to save tokens, and run test suites.
- Ideally, **success is silent, and failures are verbose.** If a typecheck passes, the agent hears nothing; if it fails, the error is injected directly back into the loop for self-correction.

Rulebook and tool choice

- **A flat markdown file at the root of a repository** is still the highest-leverage configuration point. However, it must be treated like **a pilot's checklist, not a style guide**. Keep it short, and ensure every rule is earned through a past failure.
- The same discipline applies to tools. Ten highly focused tools will always outperform fifty overlapping ones.

**O****Observability & Operations**

Tracing

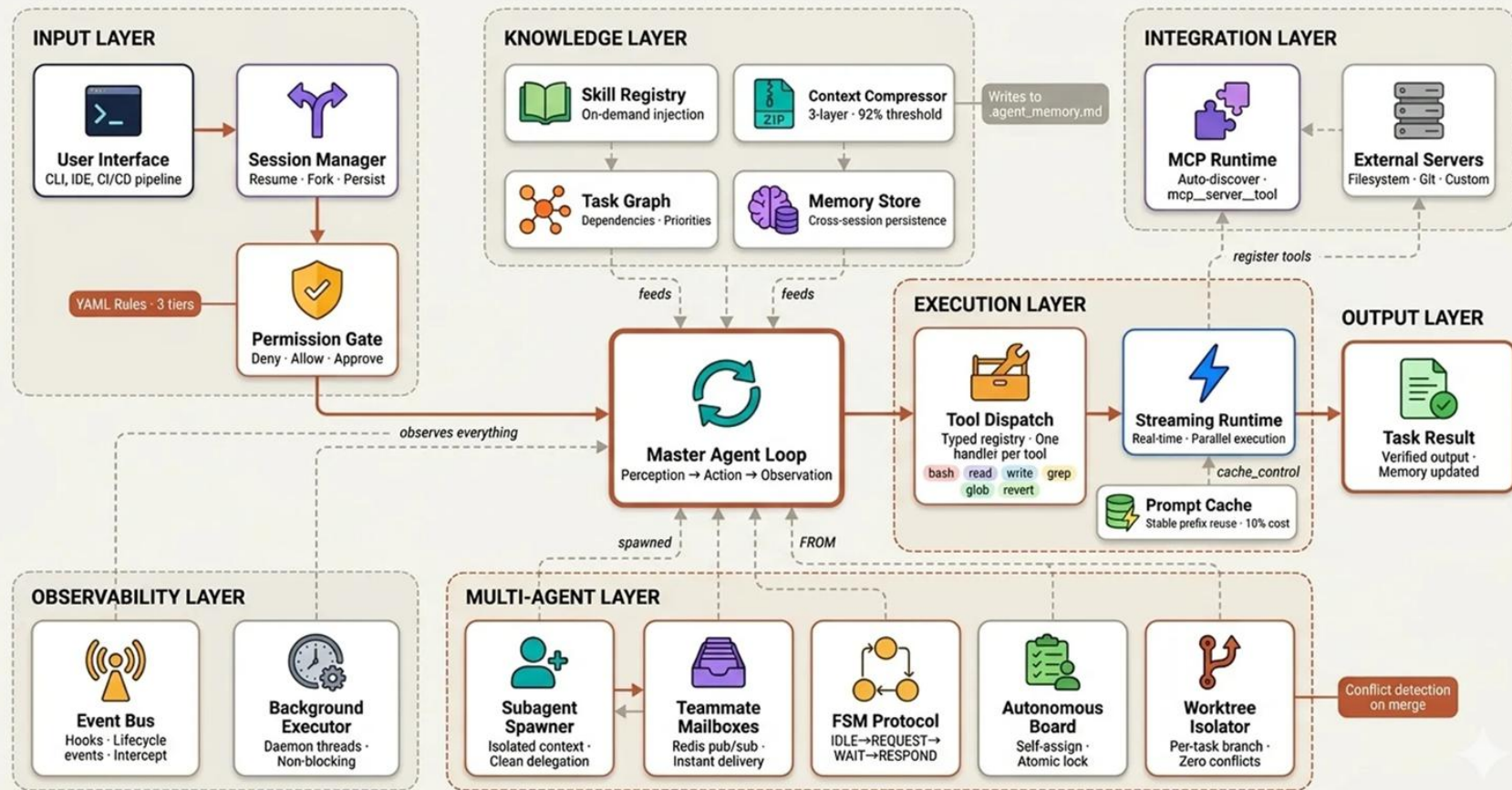
Cost and latency

Failure
diagnosisReliability
engineering**E****Execution
Environment
& Sandbox**Sandboxed runtime
Resource boundary
Execution substrate**T****Tool Interface
& Protocol**Tool schema
Protocol layer
Tool routing**C****Context
& Memory
Management**Active context
Session state
Long-term memory**L****Lifecycle
& Orchestration**Single-agent loop
Multi-agent coord.
Task pipeline**V****Verification & Evaluation**Benchmark
groundingReadiness
evaluationTrace
captureJudgement
& attributionRegression
feedback**G****Governance & Security**Permission
controlPolicy
enforcement

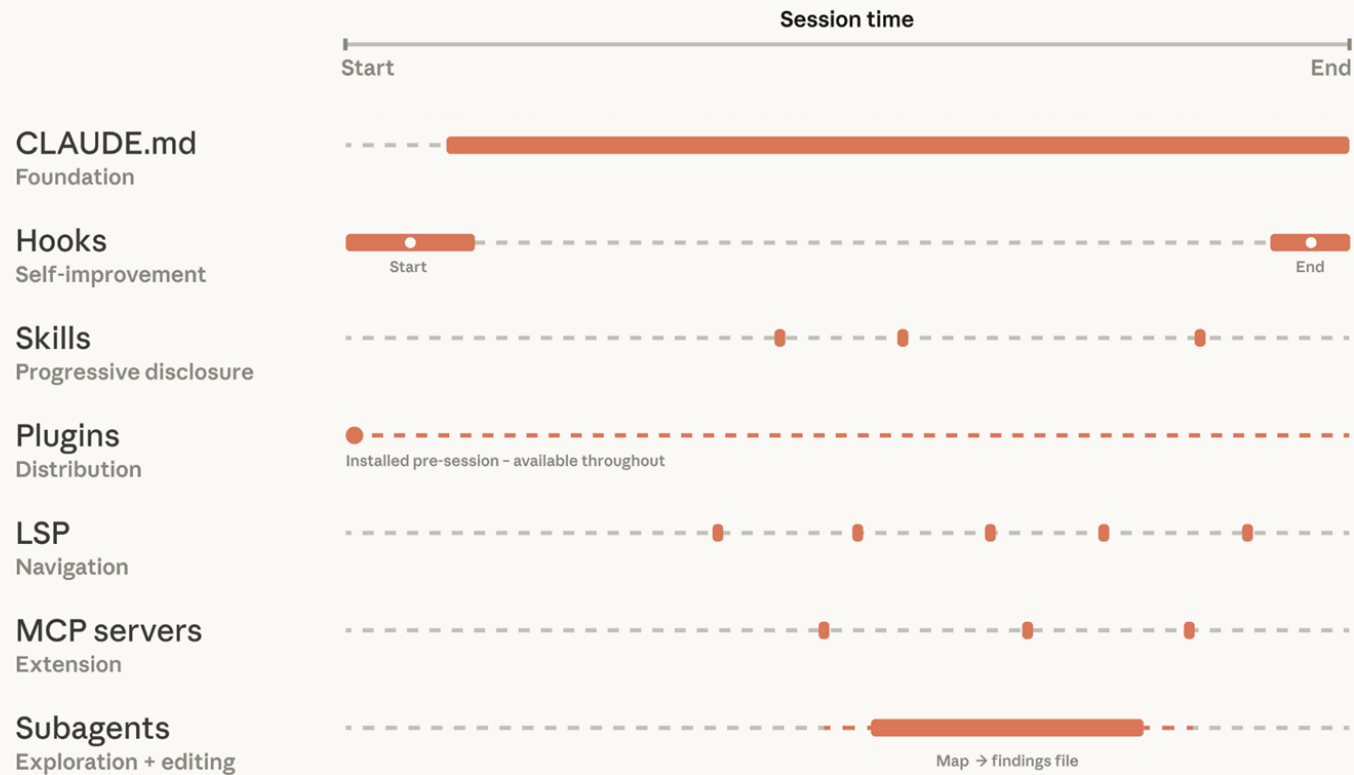
Audit trail

Security
guardrails

CLAUDE CODE ARCHITECTURE



The Claude Code harness



Harnesses Don't Shrink, They Move

- As models improve, the need for a harness doesn't disappear - it shifts.
- Every component in a harness encodes an assumption about what the model cannot do on its own. When the model improves, outdated scaffolding should be removed, and new scaffolding must be built to reach the next horizon.

The Design Principles of a Good Harness

1. **Constrain, don't instruct.** Programmatically restrict model choices; don't just rely on prompt instructions (e.g., use a schema validator, not a prompt instruction for valid JSON).
2. **Externalize state.** Critical, continuous task information (progress, failures) must be stored outside the volatile context window.
3. **Make every step verifiable.** Ensure all harness outputs can be validated by something other than the generating model. Trust requires checking.
4. **Fail locally, not globally.** A failed tool call should trigger a step-level retry, not a pipeline restart. Minimize failure impact through small-scope state management.

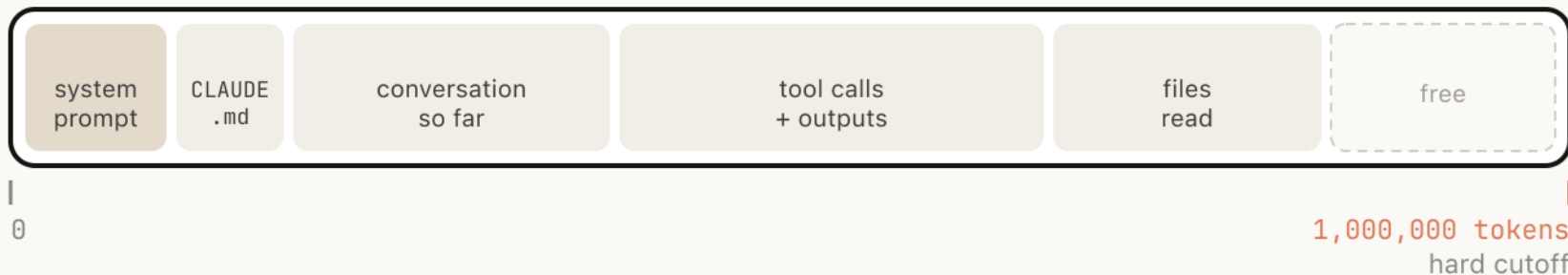
Context Management

- Recap
- Harness Engineer
- **Context Management**
- Caching
- Multi-agent patterns
- Harness Examples

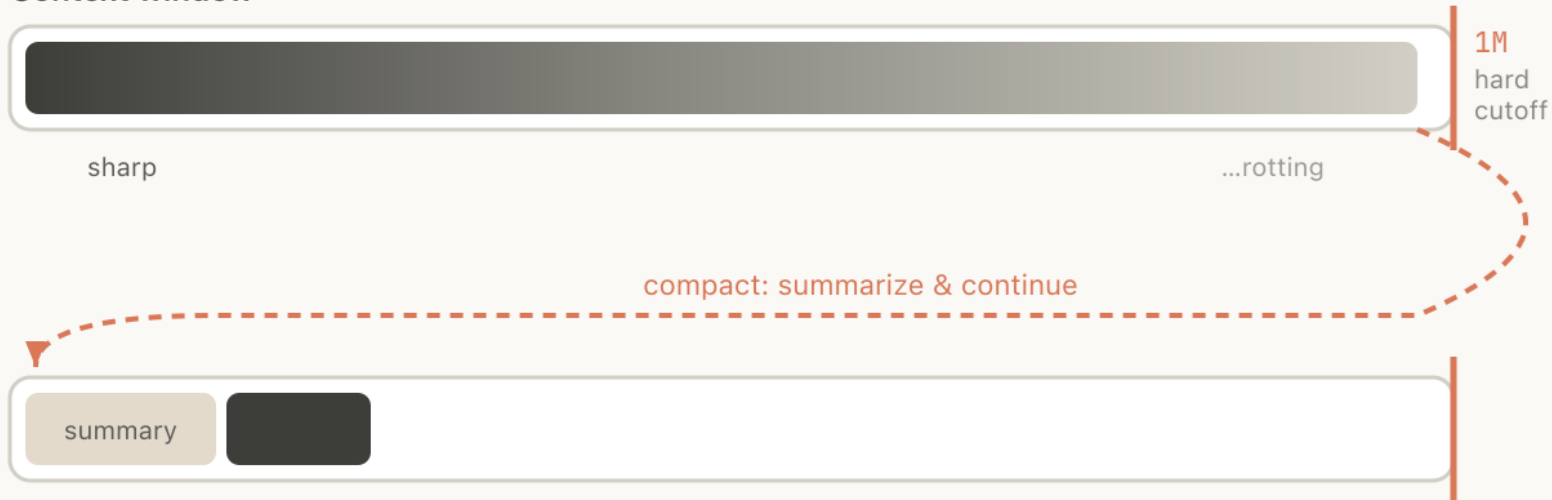
A quick primer on context, compaction and context rot

The context window

Everything the model can see when it generates the next response

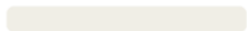


Context window



The window is finite — when it fills, compact into a summary and keep working

Fresh session



your brief only

Compact



lossy summary

Subagent



all + result

Rewind



prefix kept, tail cut

Continue



everything stays

none of it

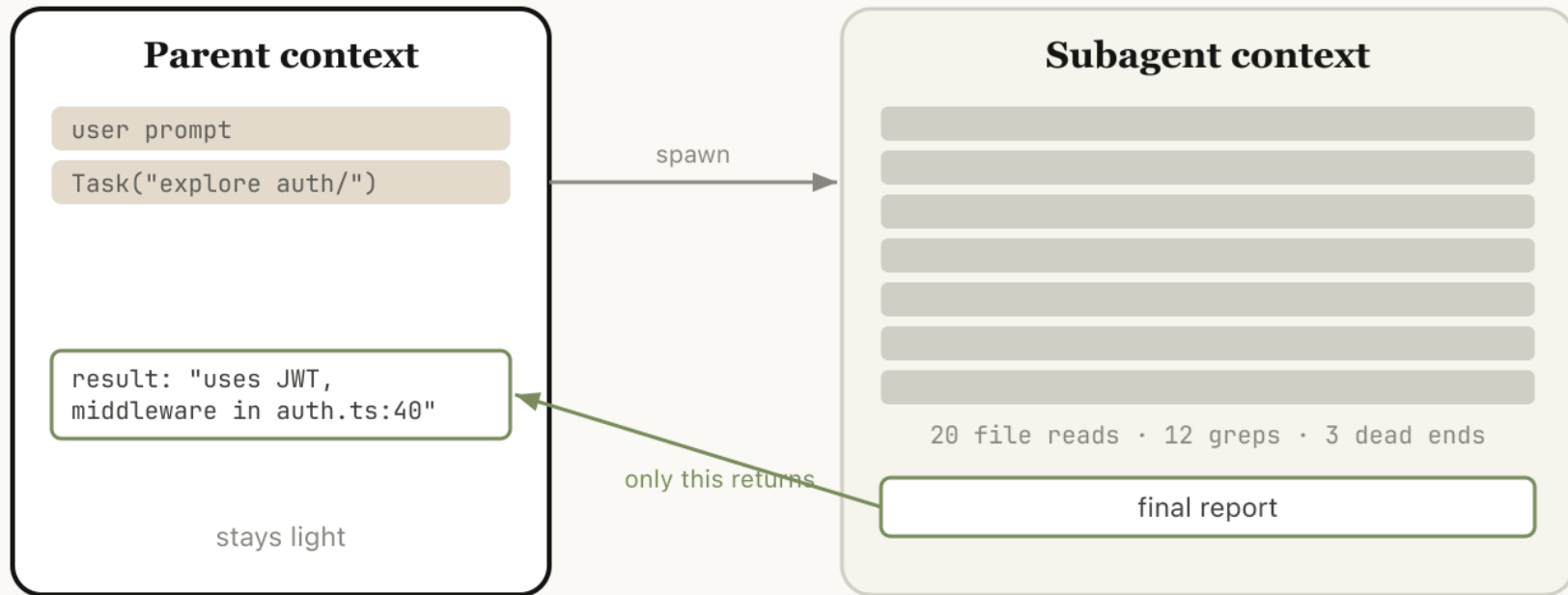
how much existing context carries forward

all of it

Compact vs. Clear



Subagents and fresh context windows



The exploration noise is garbage-collected when the subagent exits

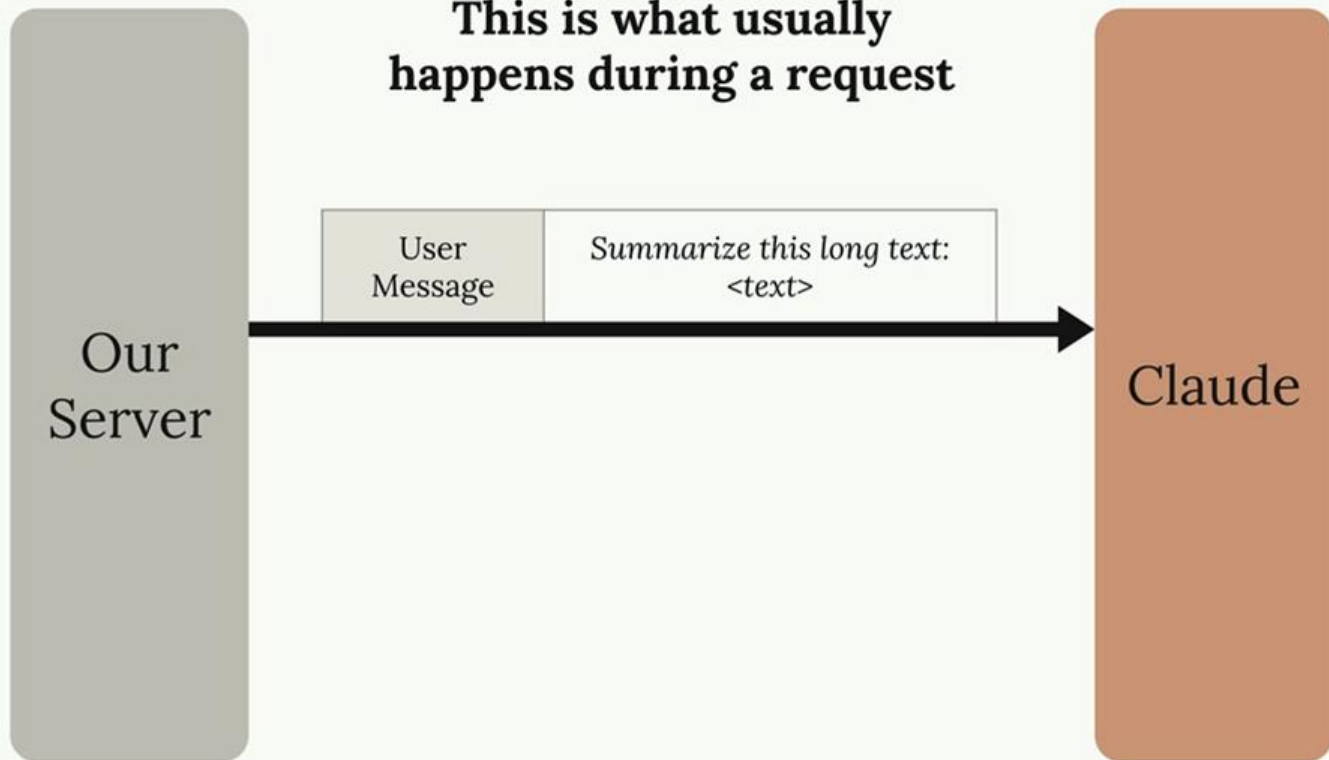
Caching

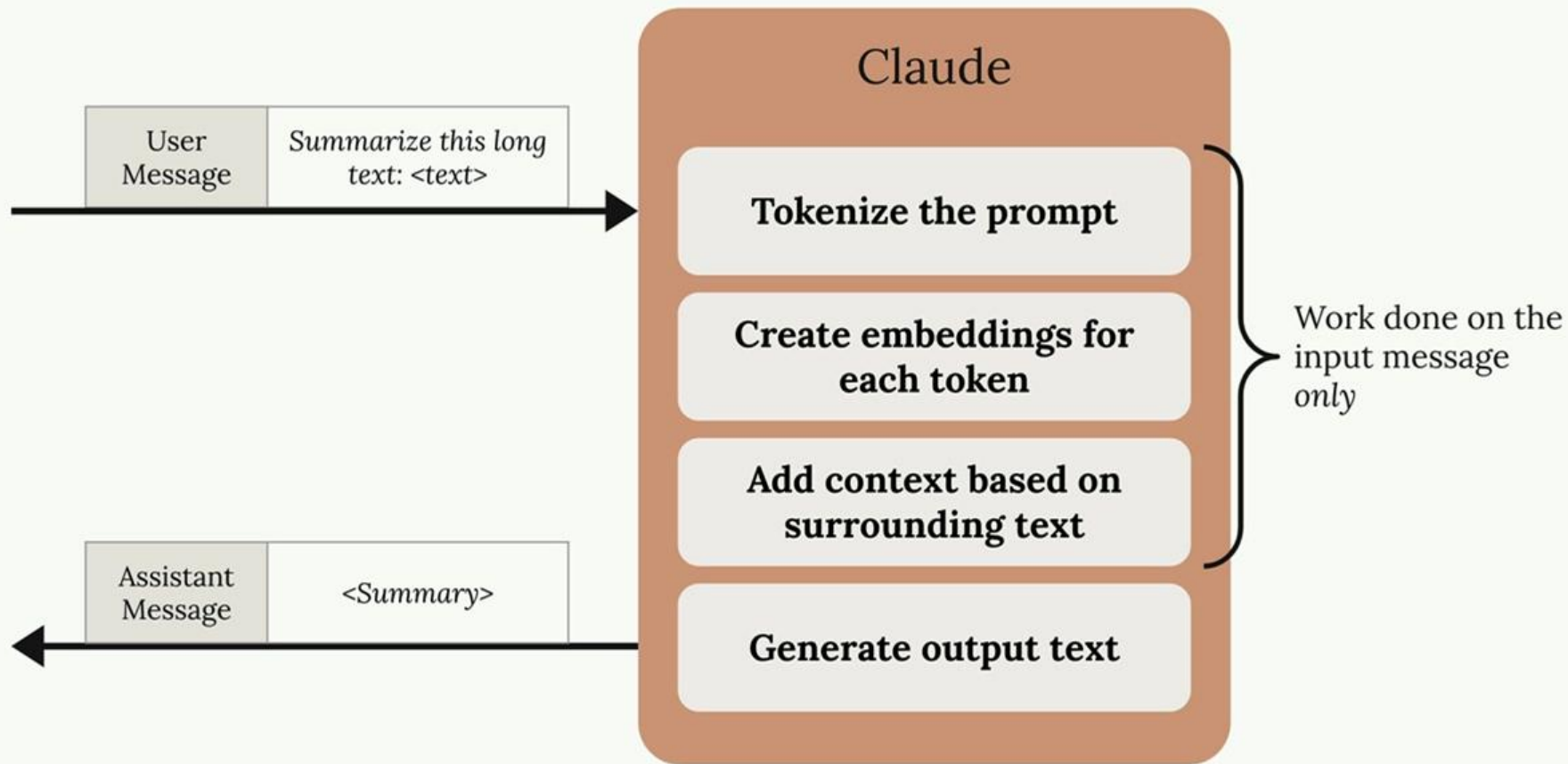
- Recap
- Harness Engineer
- Context Management
- **Caching**
- Multi-agent patterns
- Harness Examples

Prompt Caching

- **To save on inference costs and speed up responses**, you can enable **prompt caching** on supported providers and models.
- **Prompt caching**: Instead of throwing away all the processing work after each request, you can save and reuse it when you send similar content again.

**This is what usually
happens during a request**





Claude

I'm ready to process
the next request!

Trash

Tokenize the prompt

Create embeddings for each
token

Add context based on
surrounding text

Generate output text

Follow Up Request

Same message we
sent a moment ago →

User Message	Summarize this long text: <text>
Assistant Message	<Summary>
User Message	The summary needs more focus on...



Claude

Follow Up Request

User Message	Summarize this long text: <text>
Assistant Message	<Summary>
User Message	The summary needs more focus on...

Claude

Hey, wait, I just processed
that message a moment
ago!

Why did I throw away all
the work I did, I could
have reused it :(

Initial Request

User
Message

Summarize this
long text: <text>

Claude

Assistant
Message

<Summary>

Generate output text

Trash

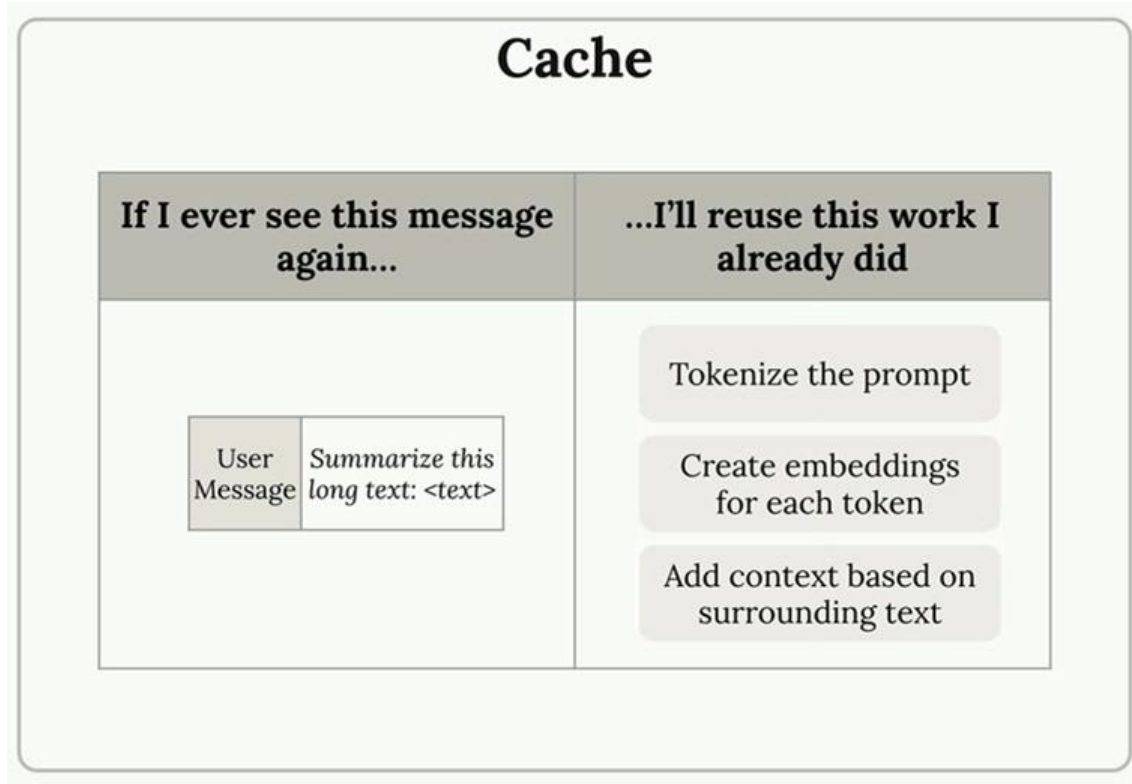
Cache

Tokenize the prompt

Create embeddings
for each token

Add context based on
surrounding text

Prompt caching saves the preprocessing work instead of discarding it



Key Benefits

Prompt caching offers several advantages:

- **Faster responses:** Requests using cached content execute more quickly
- **Lower costs:** You pay less for the cached portions of your requests
- **Automatic optimization:** The initial request writes to the cache, follow-up requests read from it

Limitations

However, there are important limitations to keep in mind:

- **Cache duration:** Cached content only lives for ***one hour*** (depending on provider)
- **Limited use cases:** Only beneficial when you're repeatedly sending the same content
- **High frequency requirement:** Most effective when the same content appears extremely frequently in your requests

Summary: Prompt Caching

- **Requests that use cached content are cheaper and faster**
- Initial request will write to the cache
- Follow up requests can read from the cache
- Cache lives for an hour
- **Only useful if you are repeatedly sending the same content**

Prompt Caching on OpenRouter

- Most providers automatically enable prompt caching, but note that some (see Anthropic below) require you to enable it on a per-message basis.
- When using caching (whether automatically in supported models, or via the `cache_control` header), OpenRouter will make a best-effort to continue routing to the same provider to make use of the warm cache. In the event that the provider with your cached prompt is not available, OpenRouter will try the next-best provider.

Prompt Caching: OpenAI

Caching price changes:

- **Cache writes:** no cost
- **Cache reads:** (depending on the model) charged at 0.25x or 0.50x the price of the original input pricing

Prompt caching with OpenAI is automated and does not require any additional configuration. There is a minimum prompt size of 1024 tokens.

Model	Input	Cached input	Output
gpt-4.1 ↳ gpt-4.1-2025-04-14	\$2.00	\$0.50	\$8.00
gpt-4.1-mini ↳ gpt-4.1-mini-2025-04-14	\$0.40	\$0.10	\$1.60
gpt-4.1-nano ↳ gpt-4.1-nano-2025-04-14	\$0.10	\$0.025	\$0.40
gpt-4.5-preview ↳ gpt-4.5-preview-2025-02-27	\$75.00	\$37.50	\$150.00
gpt-4o ↳ gpt-4o-2024-08-06	\$2.50	\$1.25	\$10.00
gpt-4o-audio-preview ↳ gpt-4o-audio-preview-2025-06-03	\$2.50	-	\$10.00
gpt-4o-realtime-preview ↳ gpt-4o-realtime-preview-2025-06-03	\$5.00	\$2.50	\$20.00

Prompt Caching: Anthropic

Caching price changes:

- **Cache writes:** charged at 1.25x the price of the original input pricing
- **Cache reads:** charged at 0.1x the price of the original input pricing

Prompt caching with Anthropic requires the use of `cache_control` breakpoints

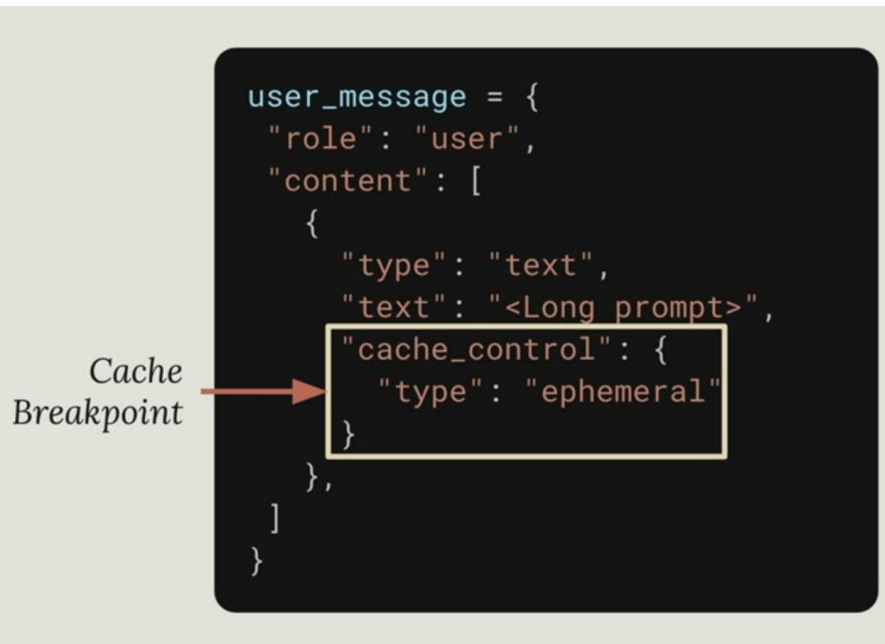
Model	Base Input Tokens	5m Cache Writes	1h Cache Writes	Cache Hits & Refreshes	Output Tokens
Claude Opus 4	\$15 / MTok	\$18.75 / MTok	\$30 / MTok	\$1.50 / MTok	\$75 / MTok
Claude Sonnet 4	\$3 / MTok	\$3.75 / MTok	\$6 / MTok	\$0.30 / MTok	\$15 / MTok
Claude Sonnet 3.7	\$3 / MTok	\$3.75 / MTok	\$6 / MTok	\$0.30 / MTok	\$15 / MTok
Claude Sonnet 3.5	\$3 / MTok	\$3.75 / MTok	\$6 / MTok	\$0.30 / MTok	\$15 / MTok
Claude Haiku 3.5	\$0.80 / MTok	\$1 / MTok	\$1.6 / MTok	\$0.08 / MTok	\$4 / MTok
Claude Opus 3	\$15 / MTok	\$18.75 / MTok	\$30 / MTok	\$1.50 / MTok	\$75 / MTok
Claude Haiku 3	\$0.25 / MTok	\$0.30 / MTok	\$0.50 / MTok	\$0.03 / MTok	\$1.25 / MTok

Note:

- 5-minute cache write tokens are 1.25 times the base input tokens price
- 1-hour cache write tokens are 2 times the base input tokens price
- Cache read tokens are 0.1 times the base input tokens price

Cache Breakpoints

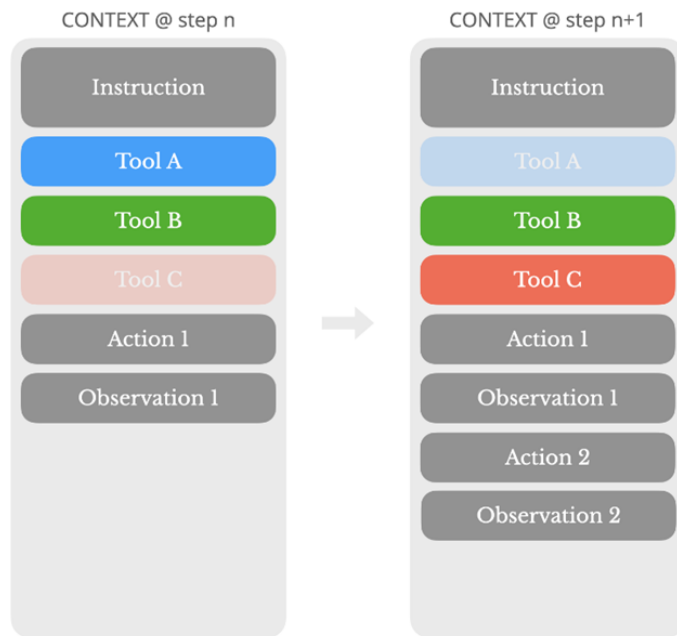
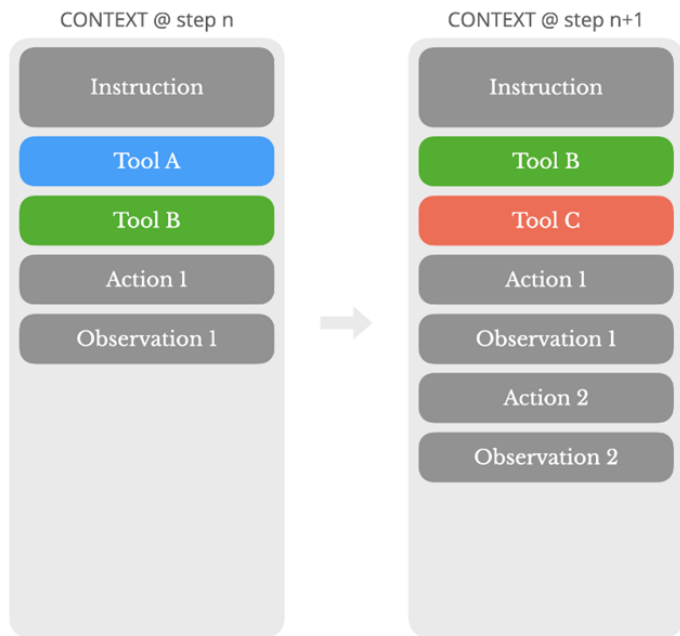
- Cache breakpoints (`cache_control`) are required for explicit prompting
- Have to manually add a 'cache breakpoint' to a block
- Work done for everything **before** the breakpoint will be cached
- Cache will be used on the follow up requests if the content up to and including the breakpoint is identical



Caching Best Practices

1. **Keep your prompt prefix stable.** A single-token difference can invalidate the cache from that token onward. A common mistake is including a time stamp.
2. **Make your context append-only.** Avoid modifying previous actions or observations. Ensure your serialization is deterministic.
3. **Make cache breakpoints explicitly when needed.** Ensure the breakpoint includes the end of system prompt.
4. **Avoid dynamically adding or removing tools mid-iteration.**
 - Tool definitions are typically before or after the system prompt. Changes will invalidate caches
 - When previous actions and observations refer to tools that are no longer defined in the context, the model get confused.

Mark, Don't Remove



Multi-agent patterns

- Recap
- Harness Engineer
- Context Management
- Caching
- **Multi-agent patterns**
- Harness Examples



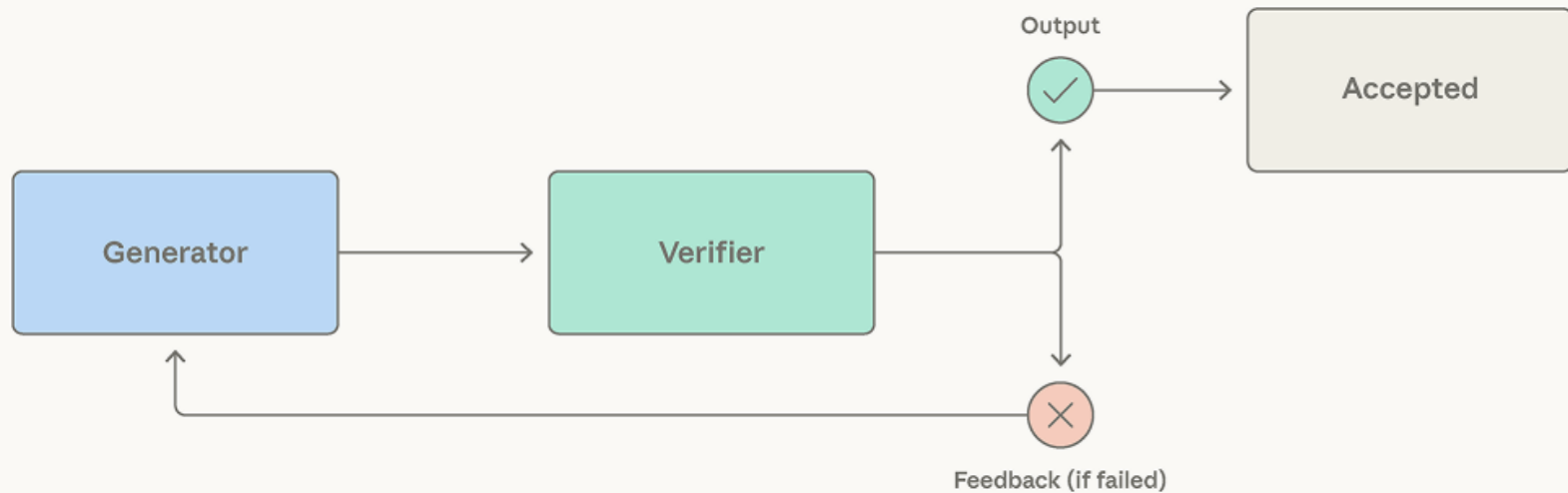
Multi-agent coordination patterns: Five approaches and when to use them

Five multi-agent coordination patterns, their trade-offs, and when to evolve from one to another.

<https://claude.com/blog/multi-agent-coordination-patterns>

Generator-Verifier

Loop until verifier accepts or max iterations reached



Generator-Verifier

Where it works well

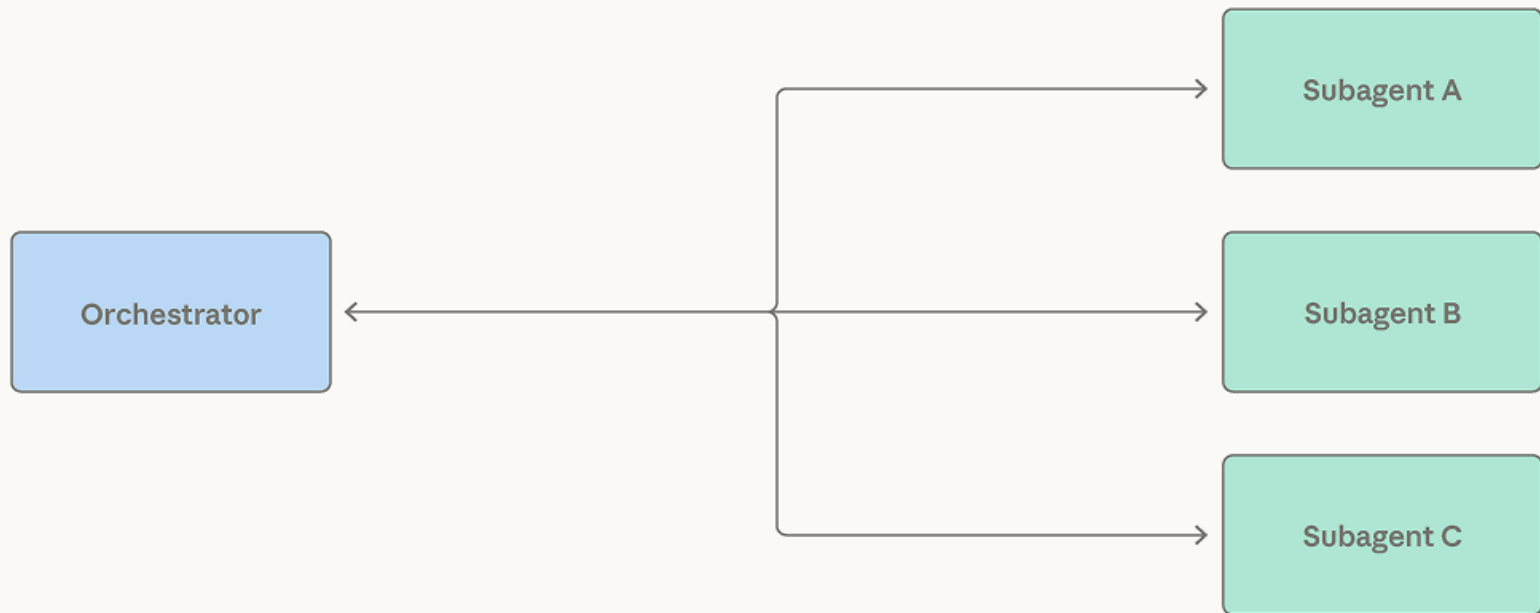
- Use when output quality is critical and evaluation criteria can be made explicit.
- Effective for code generation (one agent writes code, another writes and runs tests), fact-checking, rubric-based grading, compliance verification
- **Any domain where an incorrect output costs more than an additional generation cycle.**

Where it struggles

- The verifier is only as good as its criteria.
- If the generator can't address the verifier's feedback, the system oscillates without converging.

Orchestrator-Subagent

Orchestrator dispatches subagents and synthesizes results



Orchestrator-subagent

Where it works well

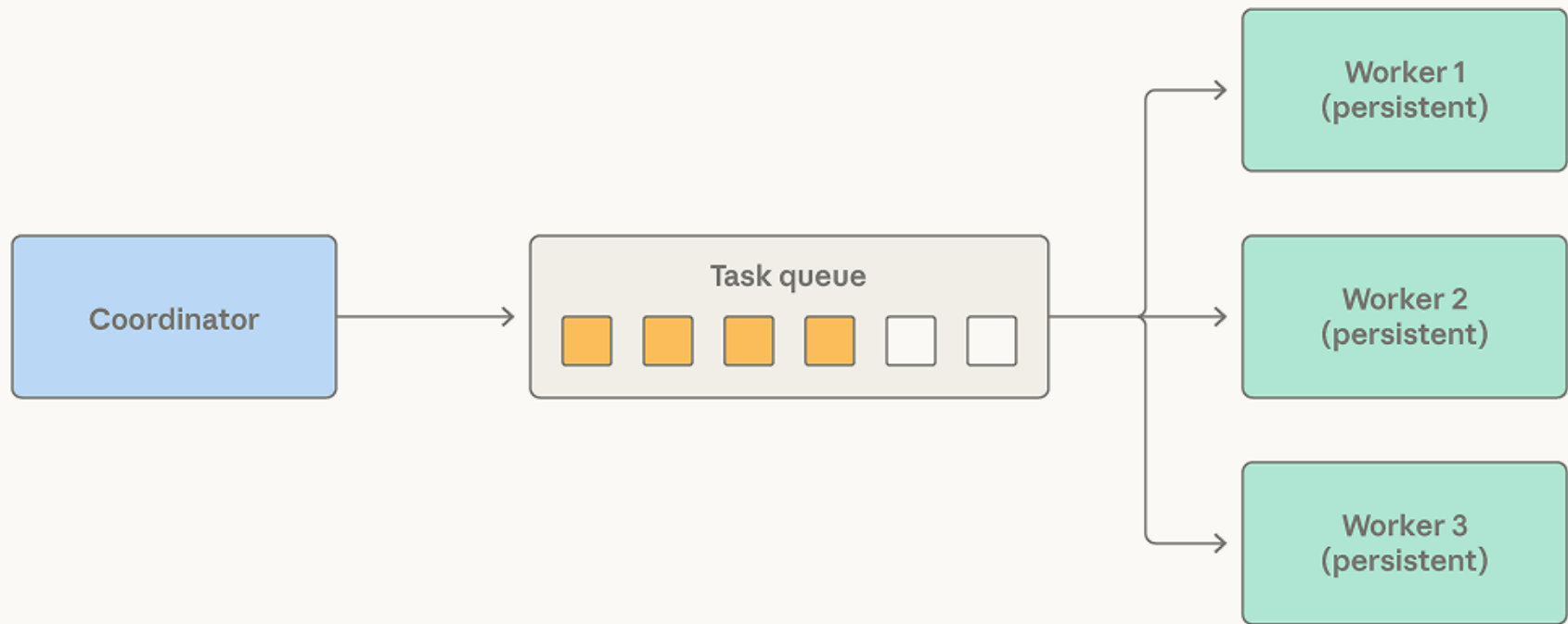
- Use this pattern when task decomposition is clear and subtasks have minimal interdependence. The orchestrator maintains a coherent view of the overall goal while subagents stay focused on specific responsibilities.

Where it struggles

- The orchestrator becomes an information bottleneck.
- When a subagent discovers something relevant to another subagent's work, that information has to travel back through the orchestrator.

Agent Teams

Workers claim tasks and work autonomously, maintaining context across steps



Agent teams

Where it works well

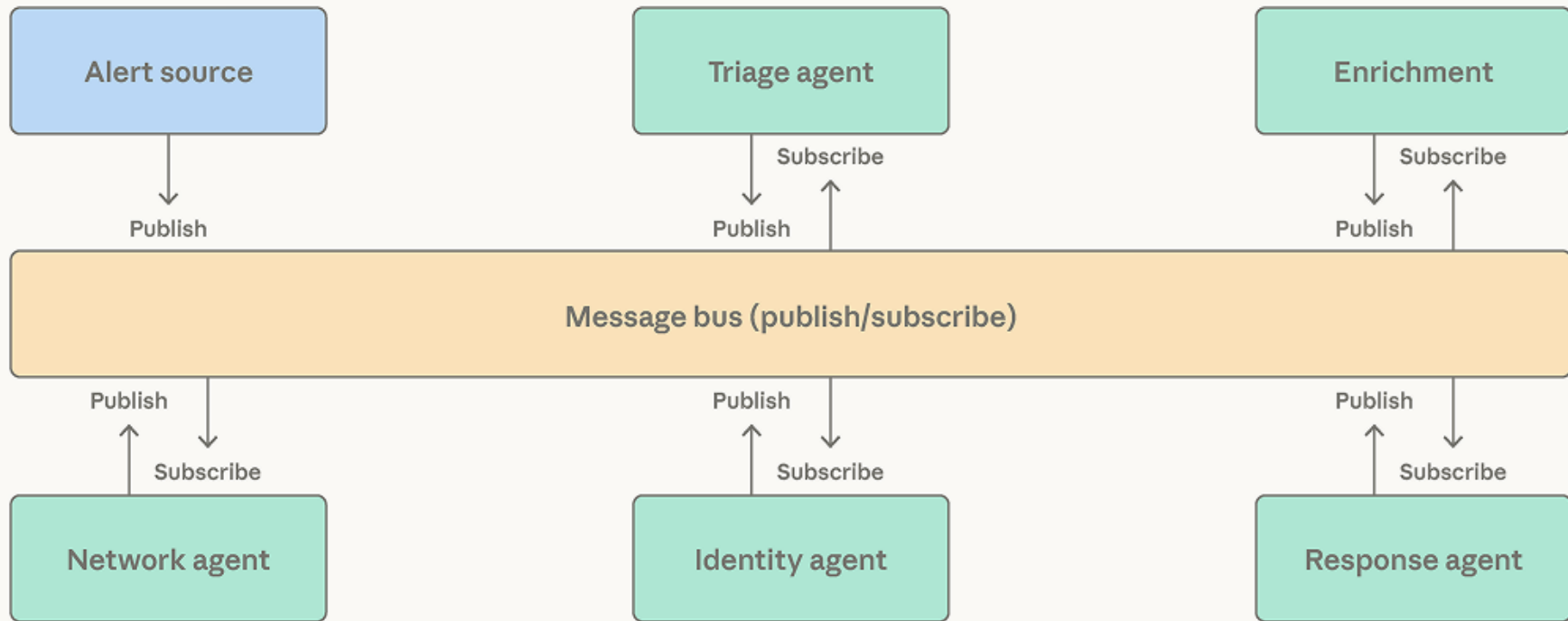
- Use this pattern when subtasks are independent and benefit from sustained, multi-step work. Each teammate builds up context about its domain rather than starting fresh with each dispatch.

Where it struggles

- Independence is the critical requirement. If one teammate's work affects another's, neither is aware, and their outputs may conflict.
- Completion detection is also harder.
- The pattern requires careful task partitioning and conflict resolution mechanisms.

Message Bus

Agents publish events and subscribe to topics they care about. Agents are decoupled.



Message Bus

Where it works well

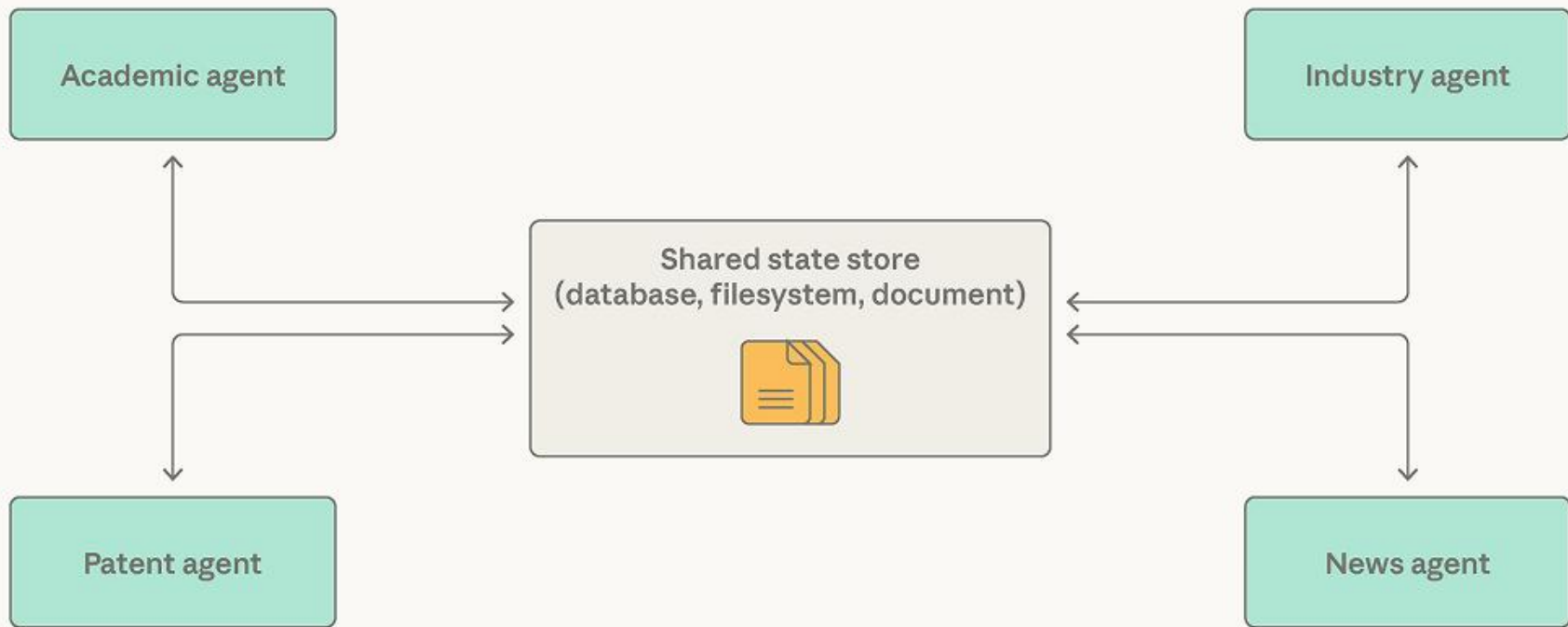
- Use this pattern for event-driven pipelines where the workflow emerges from events rather than a predetermined sequence, and where the agent ecosystem is likely to grow.

Where it struggles

- The flexibility of event-driven communication makes tracing harder.
- Routing accuracy is also critical.

Shared State

No coordinator. Agents read/write directly. Findings immediately available to all.



Shared state

Where it works well

- Agents build on each other's work, and the shared store becomes an evolving knowledge base.
- Shared state also removes the coordinator as a single point of failure.

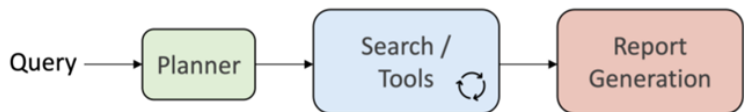
Where it struggles

- Without explicit coordination, agents may duplicate work or pursue contradictory approaches.
- The harder failure mode is reactive loops. For example, Agent A writes a finding, Agent B reads it and writes a follow-up, Agent A sees the follow-up and responds. **The system keeps burning tokens on work that isn't converging.**

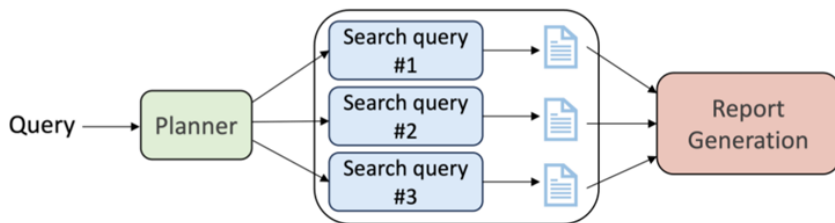
Harness Examples

- Recap
- Harness Engineer
- Context Management
- Caching
- Multi-agent patterns
- **Harness Examples**

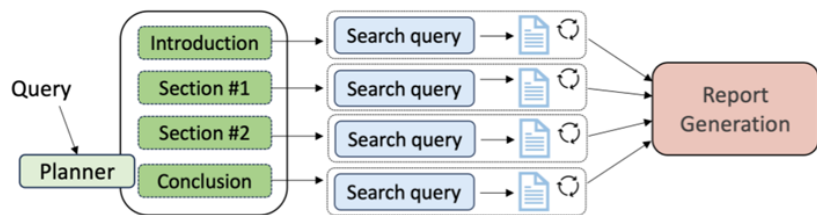
Deep Research frameworks



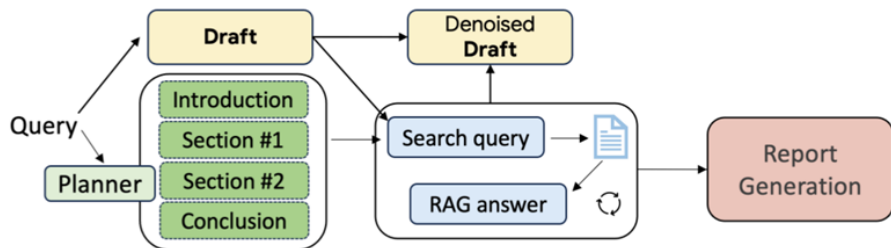
(a) Huggingface Open DR



(b) GPT Researcher

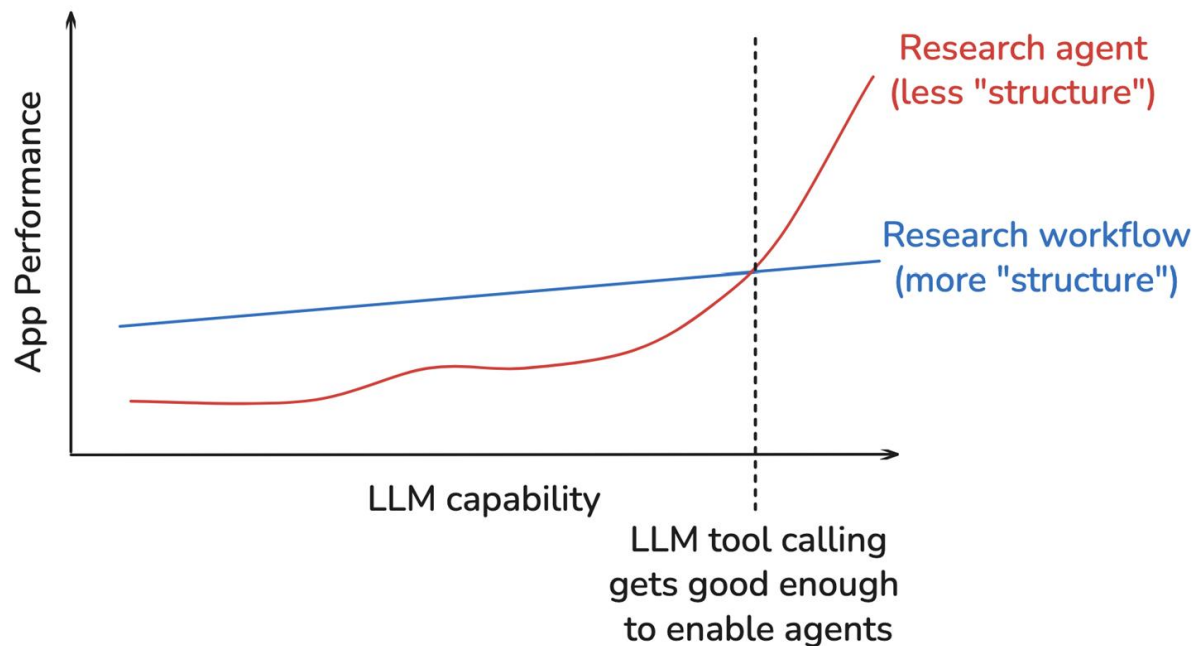


(c) Open Deep Research



(d) Test-Time Diffusion DR (ours)

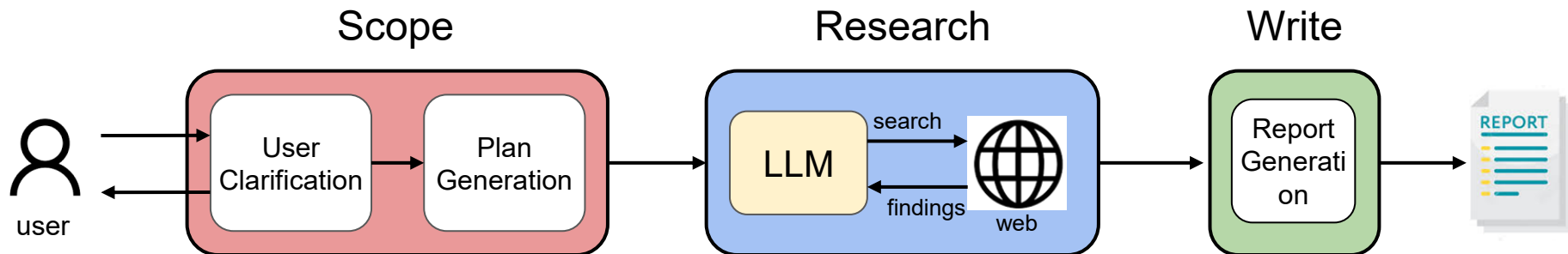
The Bitter Lesson in AI Engineering



The Bitter Lesson in AI Engineering

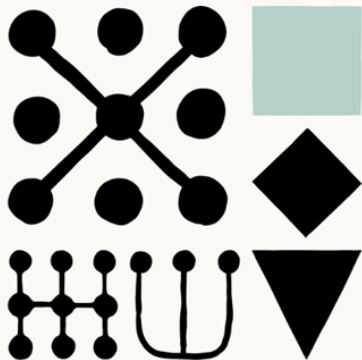
1. Understand your Harness structure.
 - a. Consider what LLM performance assumptions are baked into the design of your application.
2. Re-evaluate structure as models improve
3. Make it easy to remove structure

Web Search Agent



Multi-agent research system by Anthropic

Engineering at Anthropic



How we built our multi-agent research system

Published Jun 13, 2025

Our Research feature uses multiple Claude agents to explore complex topics more effectively. We share the engineering challenges and the lessons we learned from building this system.

<https://www.anthropic.com/engineering/multi-agent-research-system>

Multi-agent Deep Research

- **Idea: The essence of search is compression:** distilling insights from a vast corpus. Subagents facilitate compression by **operating in parallel with their own context windows**, exploring different aspects of the question simultaneously before condensing the most important tokens for the lead research agent.
- **Pros:** Dynamically finds relevant information, adapts to new findings, and analyzes results to formulate high-quality answers.
- **Cons:** Agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats.

Important factors for the performance

Three factors explained 95% of the performance variance in the BrowseComp evaluation:

- Token usage (explains 80% of the variance)
- The number of tool calls
- The model choice

For economic viability, **multi-agent systems require tasks where the value of the task is high enough to pay for the increased performance**

High-level Architecture of Advanced Research

Claude.ai chat



What's on your mind tonight?

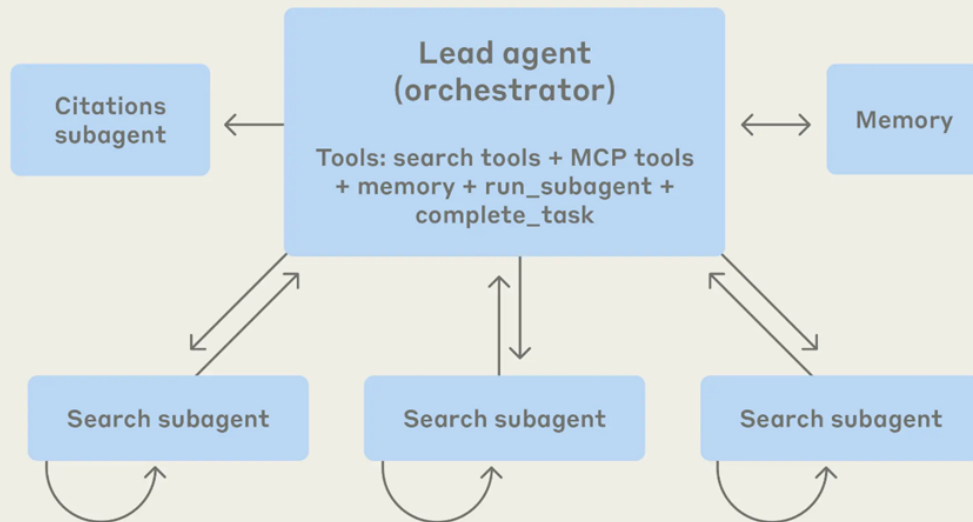


what are all the companies in the united states working on AI agents in 2025? make a list of at least 100. for each company, include the name, website, product, description of what they do, type of agents they build, and their vertical/industry.

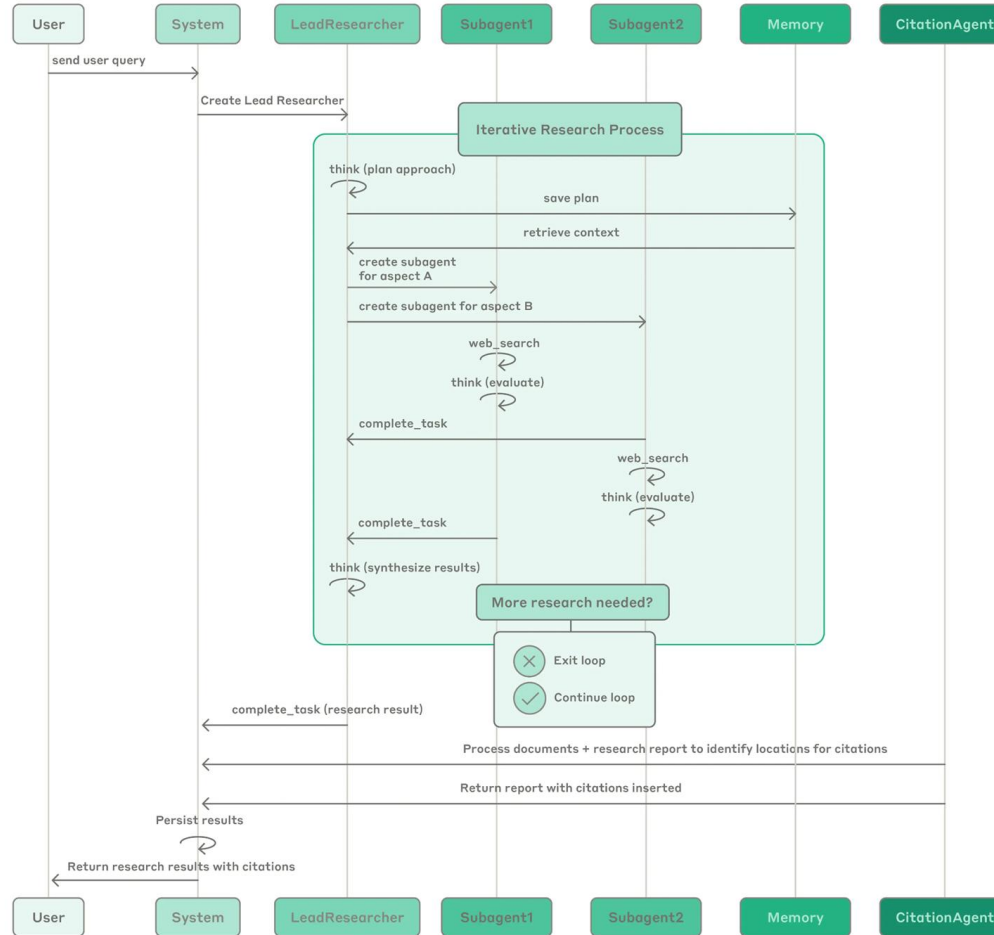
User request

Final report

Multi-agent research system



Multi-agent System Process Diagram



When you can (and can't) use multi agent

- **Key:** Each sub agent's task is **independent!**
 - Not sharing the same context
 - e.g., Agent A searches Topic X. Agent B searches Topic Y, etc.
- See limitations for coding: <https://cognition.ai/blog/dont-build-multi-agents>
- **Bottom-line:** LLMs are still not good at communicating with each other.

Prompt engineering for research agents

- **Think like your agents.** To iterate on prompts, you must understand their effects. **Effective prompting relies on developing an accurate mental model of the agent**, which can make the most impactful changes obvious.
- **Teach the orchestrator how to delegate.** In our system, the lead agent decomposes queries into subtasks and describes them to subagents. Each subagent needs an objective, an output format, guidance on the tools and sources to use, and clear task boundaries.

Prompt engineering for research agents

- **Scale effort to query complexity.** Agents struggle to judge appropriate effort for different tasks, so we embedded scaling rules in the prompts. Simple fact-finding requires just 1 agent with 3-10 tool calls, direct comparisons might need 2-4 subagents with 10-15 calls each, and complex research might use more than 10 subagents with clearly divided responsibilities.
- **Tool design and selection are critical.** We gave our agents explicit heuristics: for example, examine all available tools first, match tool usage to user intent, search the web for broad external exploration, or prefer specialized tools over generic ones.

Prompt engineering for research agents

- **Start wide, then narrow down.** Search strategy should mirror expert human research: explore the landscape before drilling into specifics.
- **Guide the thinking process.** The lead agent uses thinking to plan its approach. Subagents also plan, then use interleaved thinking after tool results to evaluate quality, identify gaps, and refine their next query
- **Parallel tool calling transforms speed and performance.**

Prompt engineering for research agents

- [Prompt for the lead agent](#)
- [Prompt for subagents](#)
- [Prompt for the citation agent](#)

Note on formatting: XML tags + Markdown

Example of reimplementatation: <https://docs.flowiseai.com/tutorials/deep-research>

Inside OpenAI's in-house data agent

By Bonnie Xu, Aravind Suresh, and Emma Tang



Listen to article

15:18

Data powers how systems learn, products evolve, and how companies make choices. But getting answers quickly, correctly, and with the right context is often harder than it should be. To make this easier as OpenAI scales, we built **our own bespoke in-house AI data agent** that explores and reasons over our own platform.

On October 6, 2025, what was our ChatGPT WAU?
Compare against our ChatGPT WAU on the last dev day in 2023. Round both to the nearest 100M.

Worked for 1m 22s >

ChatGPT WAU on October 6, 2025 (rounded to nearest 100M): $\approx$ 800M

ChatGPT WAU on the last DevDay 2023 (Nov 6, 2023; rounded to nearest 100M): $\approx$ 100M

Mini comparison (using the rounded figures):

- **Change:** $\approx$ +700M WAU
- **Multiple:** $\approx$ 8x higher on 2025-10-06 vs 2023-11-06

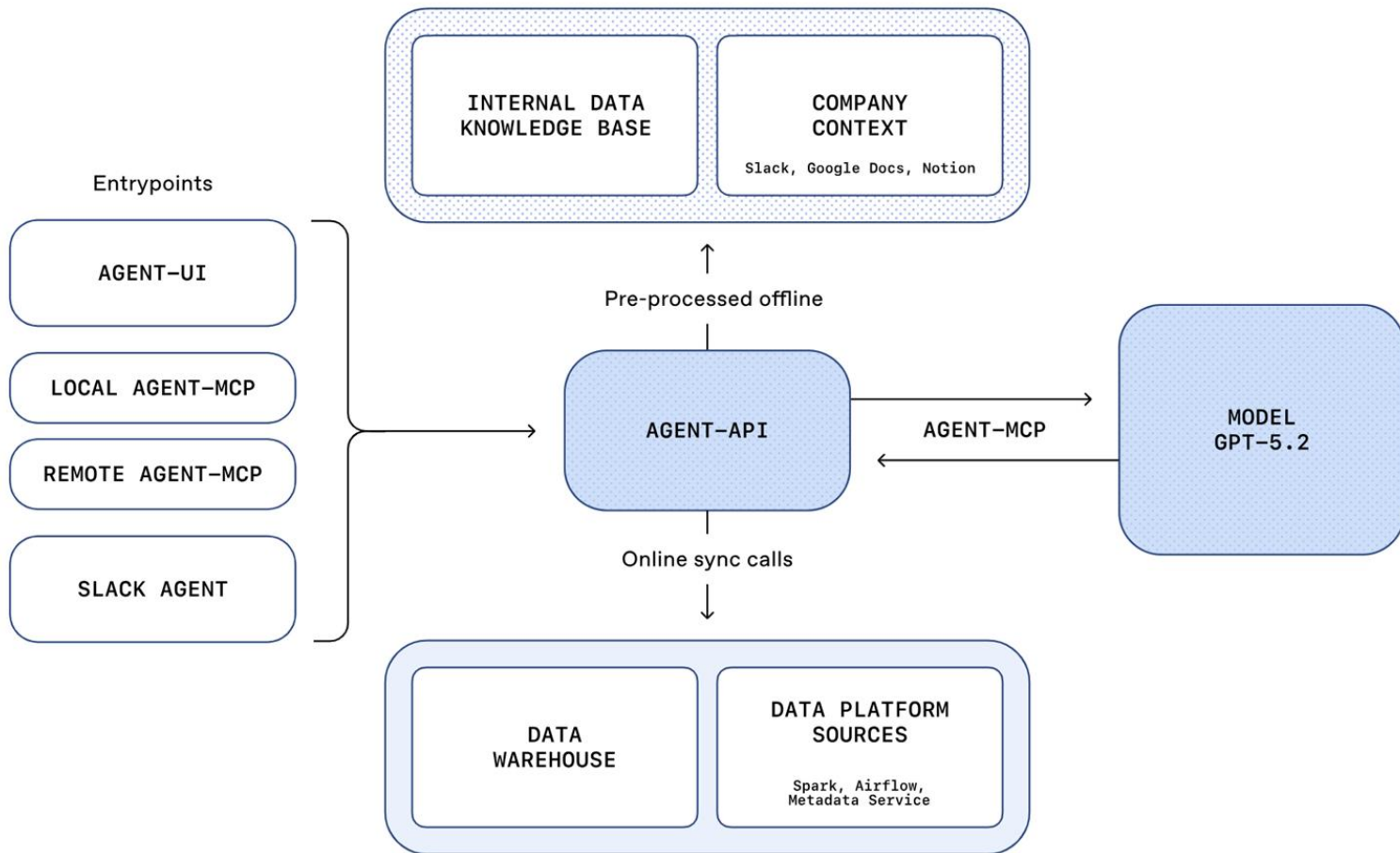
(WAU here is the standard ChatGPT WAU as-of the reporting date; I'm only sharing the values rounded to the nearest 100M, per your request.)

Why they needed a custom tool

- OpenAI's data platform serves more than **3.5k internal users** working across Engineering, Product, and Research, spanning over **600 petabytes of data** across **70k datasets**. At that size, simply finding the right table can be one of the most time-consuming parts of doing analysis.

“We have a lot of tables that are fairly similar, and I spend tons of time trying to figure out how they're different and which to use. Some include logged-out users, some don't. Some have overlapping fields; it's hard to tell what is what.”

How the data agent works



Context is everything

High-quality answers depend on rich, accurate context.

What was ChatGPT Image Gen logged-in DAU for the last 30 days?

Worked for 22m 41s >

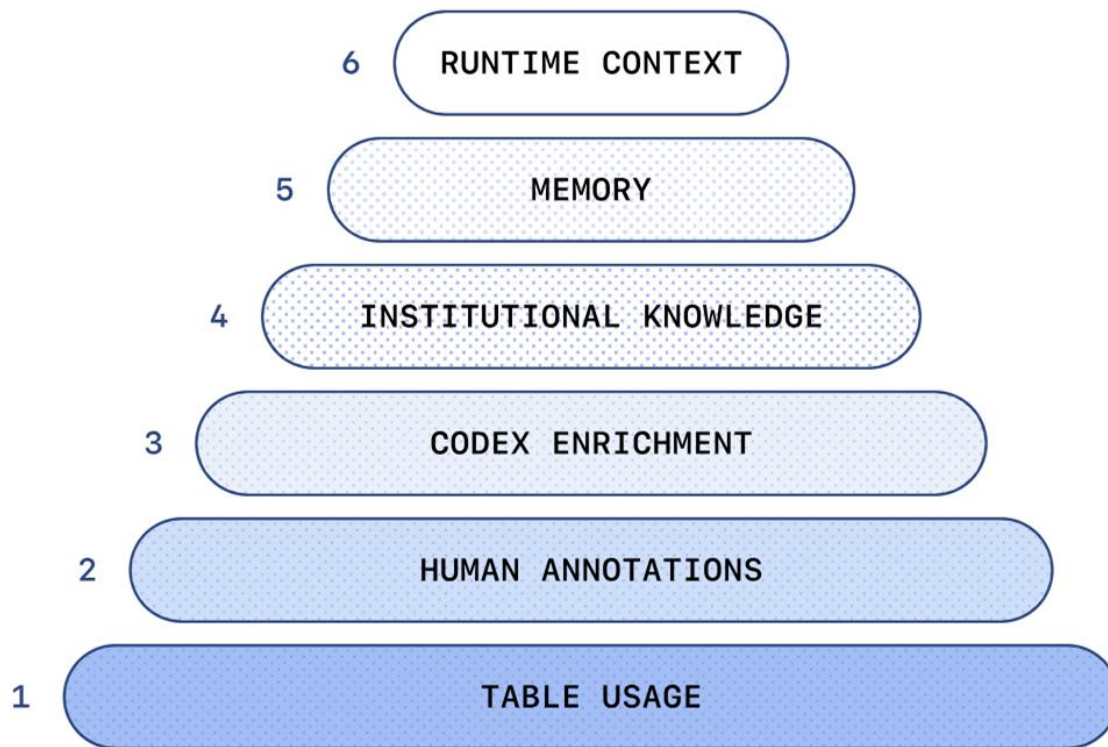
The agent without memory, unable to query effectively.

What was ChatGPT Image Gen logged-in DAU for the last 30 days?

Worked for 1m 22s >

The agent's memory enables faster queries by locating the correct tables.

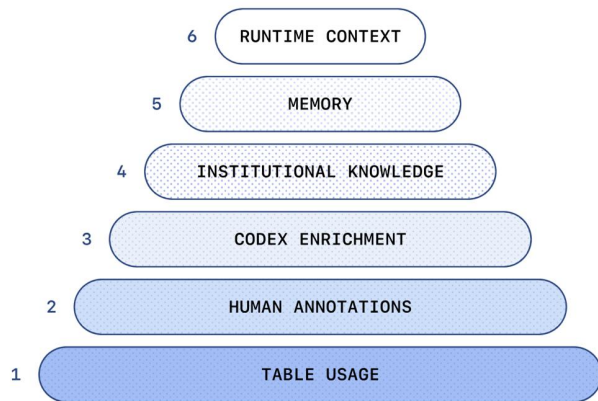
Data agent's layers of context



Layer #1: Table Usage

- **Metadata grounding:** The agent relies on schema metadata (column names and data types) to inform SQL writing and uses table lineage (e.g., upstream and downstream table relationships) to provide context on how different tables relate.
- **Query inference:** Ingesting historical queries helps the agent understand how to write its own queries and which tables are typically joined together.

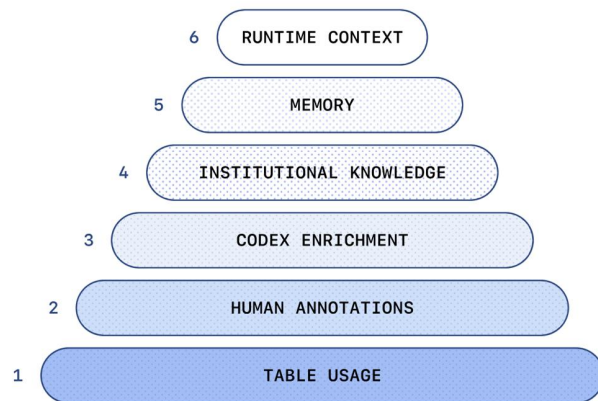
Data agent's layers of context



Layer #2: Human Annotations

- **Curated descriptions** of tables and columns provided by domain experts, capturing intent, semantics, business meaning, and known caveats that are not easily inferred from schemas or past queries.

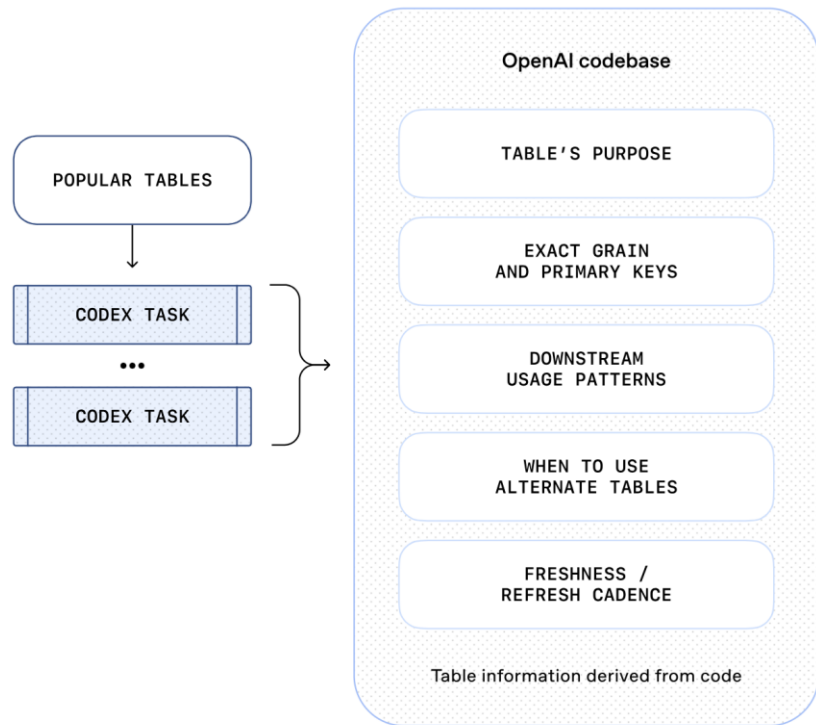
Data agent's layers of context



Layer #3: Codex Enrichment

- By defining tables at the code level, the agent gains a deeper understanding of the data's content, including nuances like value uniqueness, update frequency, and data scope.
- This also provides enhanced usage context beyond SQL.
- So, the agent can differentiate between similar-looking tables (e.g., distinguishing tables by traffic source like first-party ChatGPT).

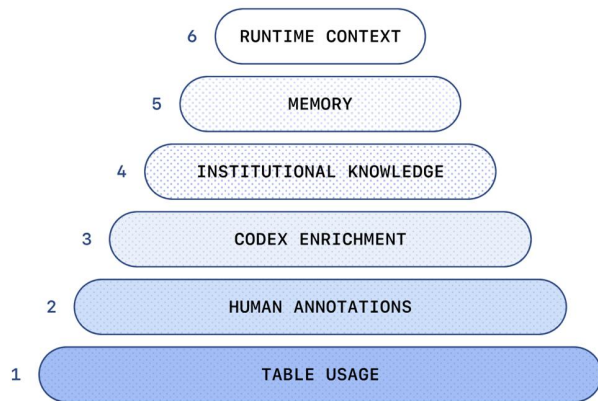
Codex-enriched knowledge pipeline



Layer #4: Institutional Knowledge

- The agent accesses critical company context (launches, incidents, codenames, metrics, etc.) from Slack, Google Docs, and Notion.
- These documents are ingested, embedded, and stored with metadata and permissions. A retrieval service ensures efficient and safe access control and caching at runtime.

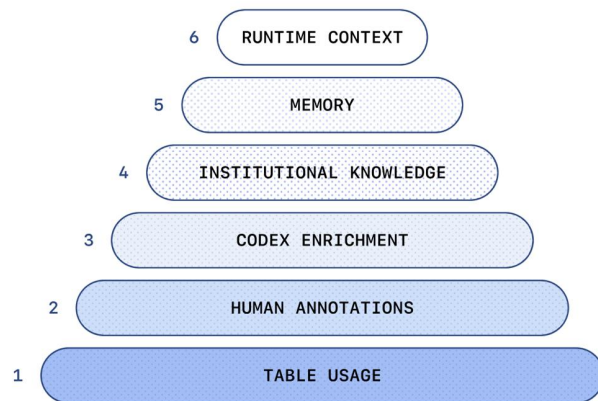
Data agent's layers of context



Layer #5: Memory

- When the agent is given corrections or discovers nuances about certain data questions, it's able to save these learnings for next time, allowing it to constantly improve with its users.
- When you give the agent a correction or when it finds a learning from your conversation, it will prompt you to save that memory for next time.

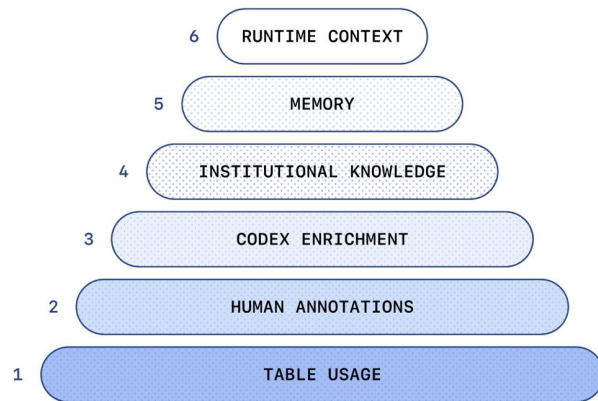
Data agent's layers of context



Layer #6: Runtime Context

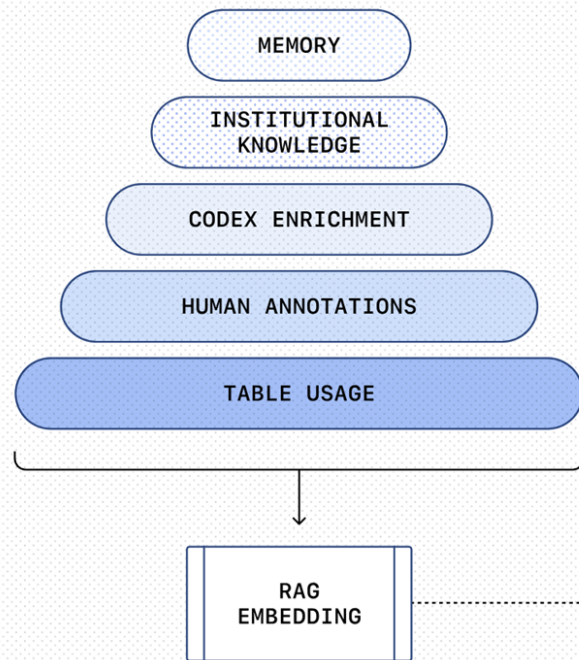
- If a table lacks context or has stale information, the agent can query the data warehouse directly. This allows real-time schema validation, data understanding, and appropriate responses.
- The agent can also access other Data Platform systems (metadata service, Airflow, Spark) for broader context outside the warehouse.

Data agent's layers of context

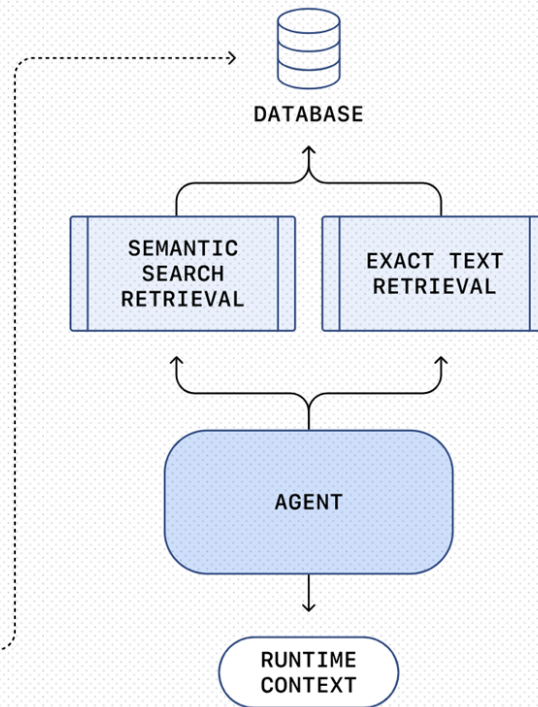




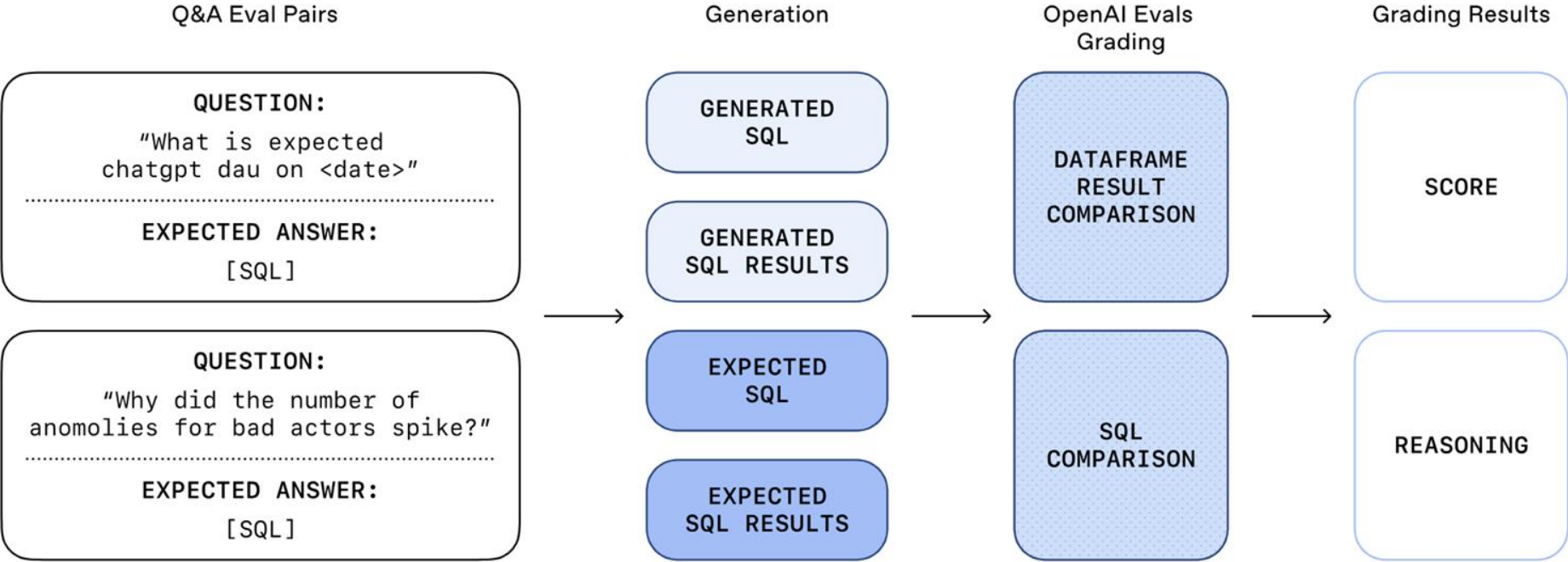
Offline job pre-processing



Live retrieval



Data agent's evaluation pipeline



Lessons learned

Lesson #1: Less is More: Exposing the agent's full tool set caused issues with overlapping functionality, which confused the agent. We restricted and consolidated tool calls to reduce ambiguity and improve reliability.

Lesson #2: Guide the Goal, Not the Path: Highly prescriptive prompting degraded results by pushing the agent down incorrect paths. Shifting to higher-level guidance, relying on GPT-5's reasoning, made the agent more robust and produced better results.

Lesson #3: Meaning Lives in Code: While schemas and query history describe tables, their true meaning is in the underlying code (pipeline logic, assumptions, etc.). By crawling the codebase with Codex, our agent understands dataset construction, allowing it to accurately reason about table content and usage.

Harness engineering: leveraging Codex in an agent-first world

By Ryan Lopopolo, Member of the Technical Staff



Listen to article

17:57



Share

Over the past five months, our team has been running an experiment: building and shipping an internal beta of a software product with **0 lines of manually-written code**.

The product has internal daily users and external alpha testers. It ships, deploys, breaks, and gets fixed. What's different is that every line of code—application logic, tests, CI configuration, documentation, observability, and internal tooling—has been written by Codex. We estimate that we built this in about 1/10th the time it would have taken to write the code by hand.

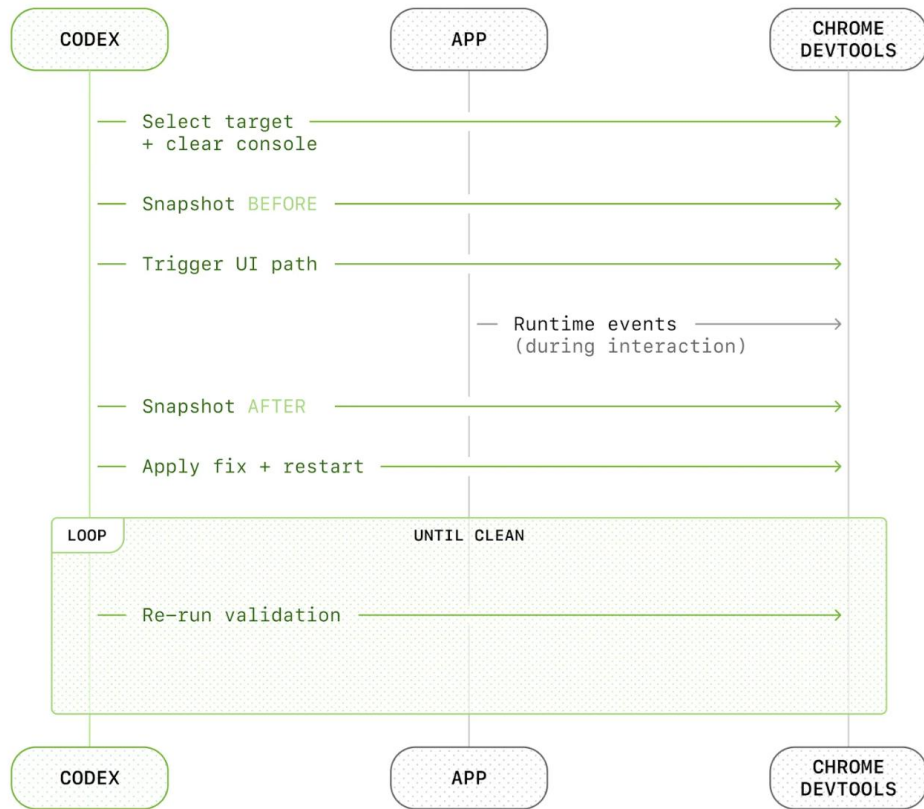
Redefining the role of the engineer

- The lack of hands-on human coding introduced a different kind of engineering work, focused on systems, scaffolding, and leverage.
- When something failed, the fix was almost never “try harder.” Because the only way to make progress was to get Codex to do the work, human engineers always stepped into the task and asked: “what capability is missing, and how do we make it both legible and enforceable for the agent?”

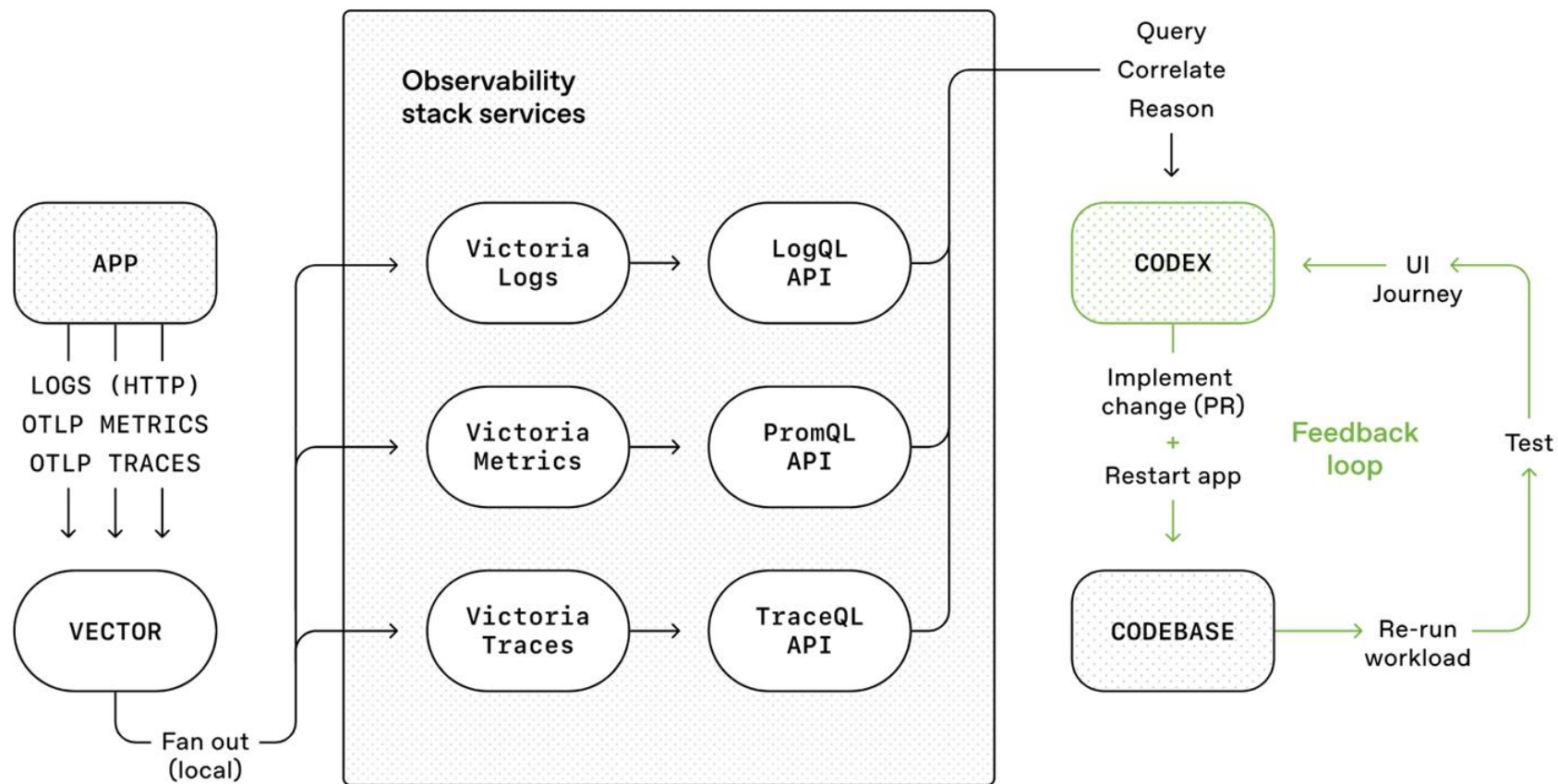
Increasing application legibility

As code throughput increased, our bottleneck became human QA capacity. Because the fixed constraint has been human time and attention, we've worked to add more capabilities to the agent by making things like the application UI, logs, and app metrics themselves directly legible to Codex.

Codex drives the app with Chrome DevTools MCP to validate its work



Giving Codex a full observability stack



We made repository knowledge the system of record

- **Give Codex a map, not a 1,000-page instruction manual.**
- One big AGENTS.md failed in predictable ways:
 - **Context is a scarce resource.** A giant instruction file crowds out the task, the code, and the relevant docs. The agent either misses key constraints or starts optimizing for the wrong ones.
 - **Too much guidance becomes non-guidance.** When everything is “important,” nothing is. Agents end up pattern-matching locally instead of navigating intentionally.
 - **It rots instantly.** A monolithic manual turns into a graveyard of stale rules. Agents can’t tell what’s still true, humans stop maintaining it, and the file quietly becomes an attractive nuisance.
 - **It’s hard to verify.** A single blob doesn’t lend itself to mechanical checks (coverage, freshness, ownership, cross-links), so drift is inevitable.

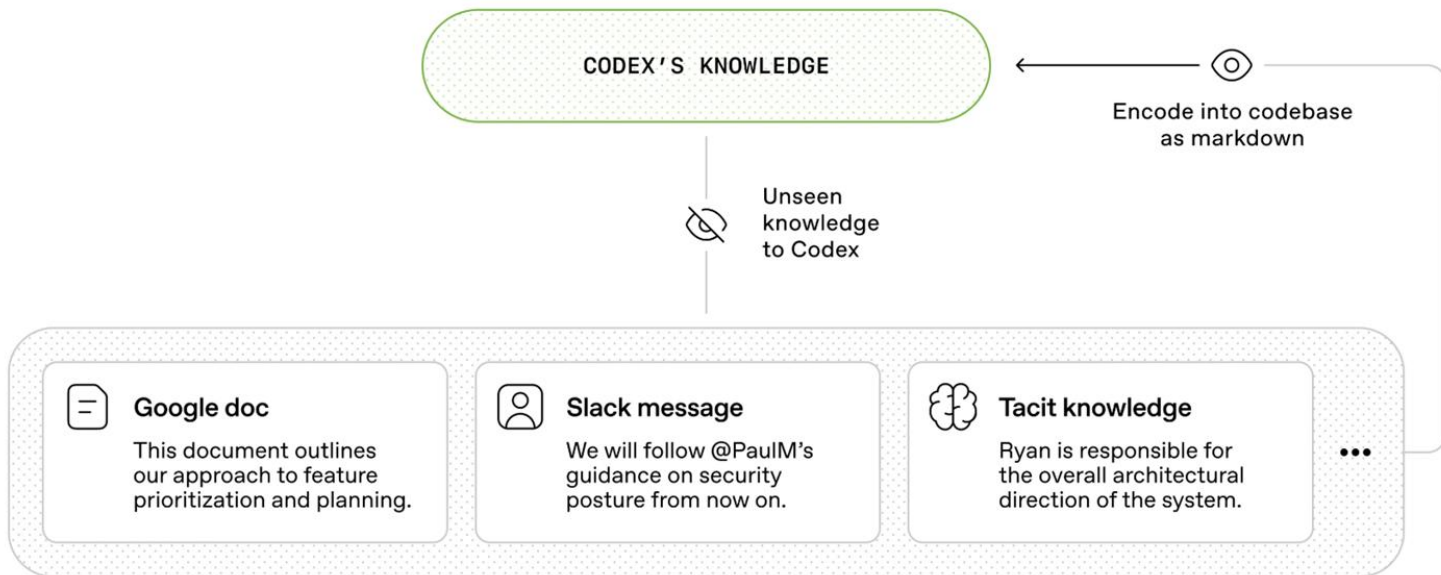
AGENTS.md = Table of contents.

- **Design documentation** is indexed with verification status and core agent-first principles.
- **Architecture documentation** maps domains and package layering.
- **Plans are first-class artifacts**: small changes use ephemeral plans; complex work uses execution plans with progress/decision logs in the repository.
- This allows **progressive disclosure**: agents begin at a stable entry point and are guided, avoiding overwhelm.

```
1 AGENTS.md
2 ARCHITECTURE.md
3 docs/
4 |─ design-docs/
5 |   |─ index.md
6 |   |─ core-beliefs.md
7 |   └─ ...
8 |─ exec-plans/
9 |   |─ active/
10 |   |─ completed/
11 |   └─ tech-debt-tracker.md
12 |─ generated/
13 |   └─ db-schema.md
14 |─ product-specs/
15 |   |─ index.md
16 |   |─ new-user-onboarding.md
17 |   └─ ...
18 |─ references/
19 |   |─ design-system-reference-llms.txt
20 |   |─ nixpacks-llms.txt
21 |   |─ uv-llms.txt
22 |   └─ ...
23 |─ DESIGN.md
24 |─ FRONTEND.md
25 |─ PLANS.md
26 |─ PRODUCT_SENSE.md
```

Agent legibility is the goal

The limits of agent knowledge:
What Codex can't see doesn't exist



Entropy and garbage collection

- Codex replicates patterns that already exist in the repository including suboptimal ones. Over time, this inevitably leads to drift.
- Initially, humans addressed this manually, spending every Friday cleaning up “AI slop.” Unsurprisingly, that didn’t scale.
- We started encoding what we call “golden principles” directly into the repository and built a recurring cleanup process, which help keep the codebase legible and consistent for future agent runs.
- This functions like garbage collection. **Technical debt is like a high-interest loan: it’s almost always better to pay it down continuously in small increments than to let it compound and tackle it in painful bursts.**

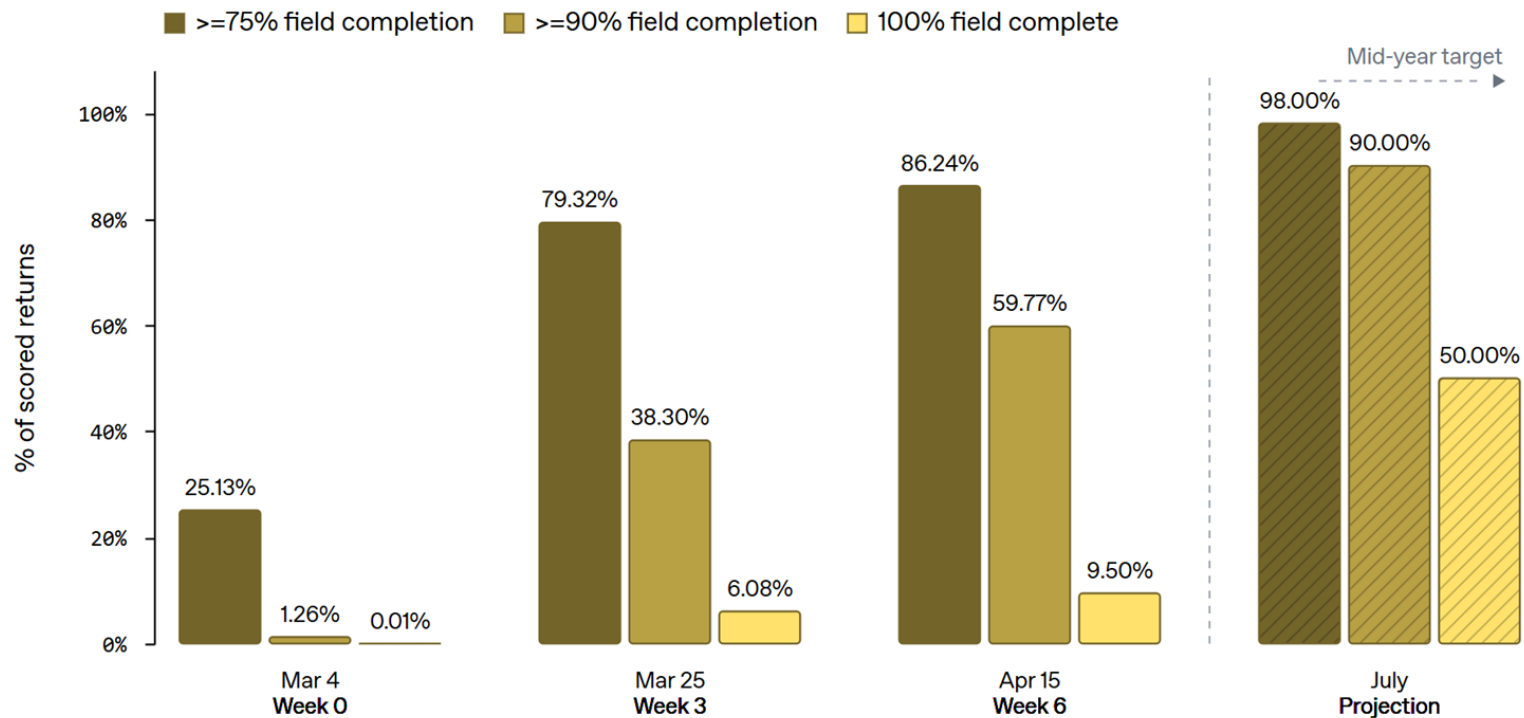
May 27, 2026 Engineering

Building self-improving tax agents with Codex

By Members of Technical Staff: Aravind Srinivasan & Samay Shamdasani (Thrive Holdings), Arthur Fernandes Araujo & John de Wasseige (OpenAI)

<https://openai.com/index/building-self-improving-tax-agents-with-codex/>

Tax AI completion accelerated through tax season



As Tax AI expanded into more complex filings, the share of scored returns reaching 75%, 90%, and full completion continued to rise through tax season.

The problem

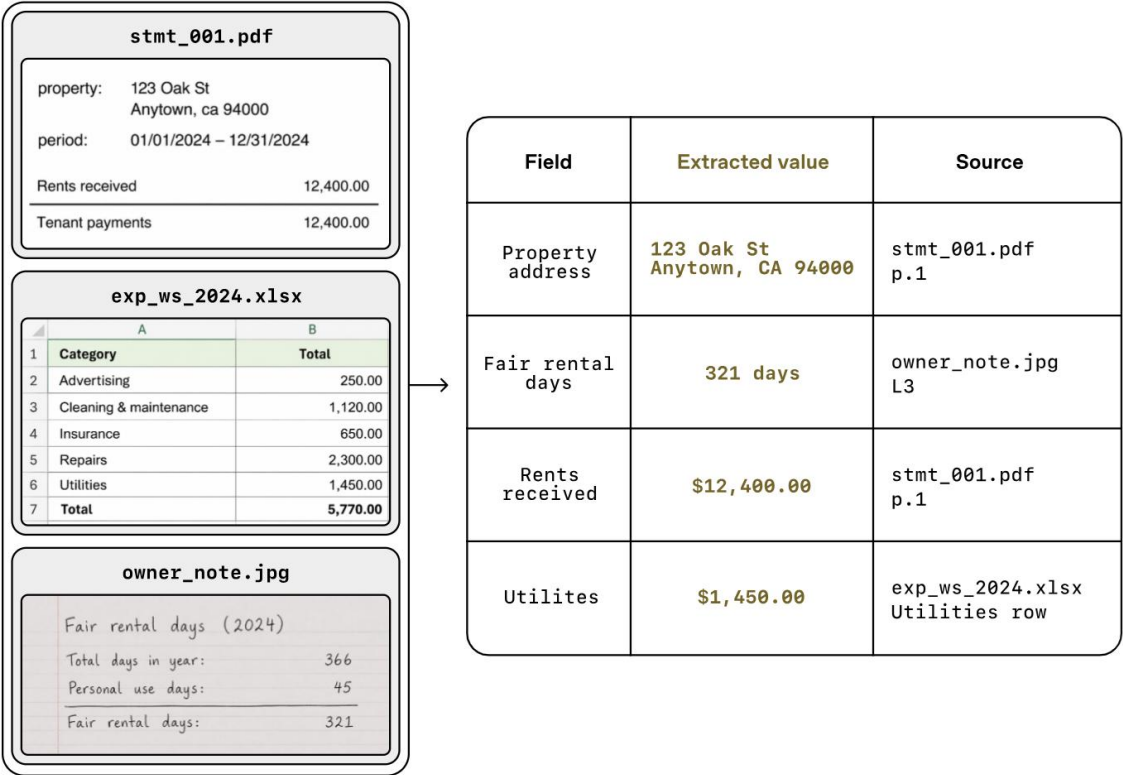
As we pushed into harder parts of tax preparation (K-1s, rental real estate schedules, and tax forms where values had to be reconciled across multiple source files), it became obvious that the real challenge was whether the product could make complex production failures visible, understandable, and actionable.

In the early days of the product, most of the correction was manual. Practitioners could correct system errors, but the product did not capture the full context: a changed value before filing might reflect a true extraction miss, a mapping problem, missing product support, or expected workflow noise. Sorting those cases out still required follow-up from the engineering team. Engineers could use coding agents, but the system was not yet designed to use AI meaningfully inside an improvement loop. We did not have the signal to identify the right hill to climb.

Our approach: a three-part loop

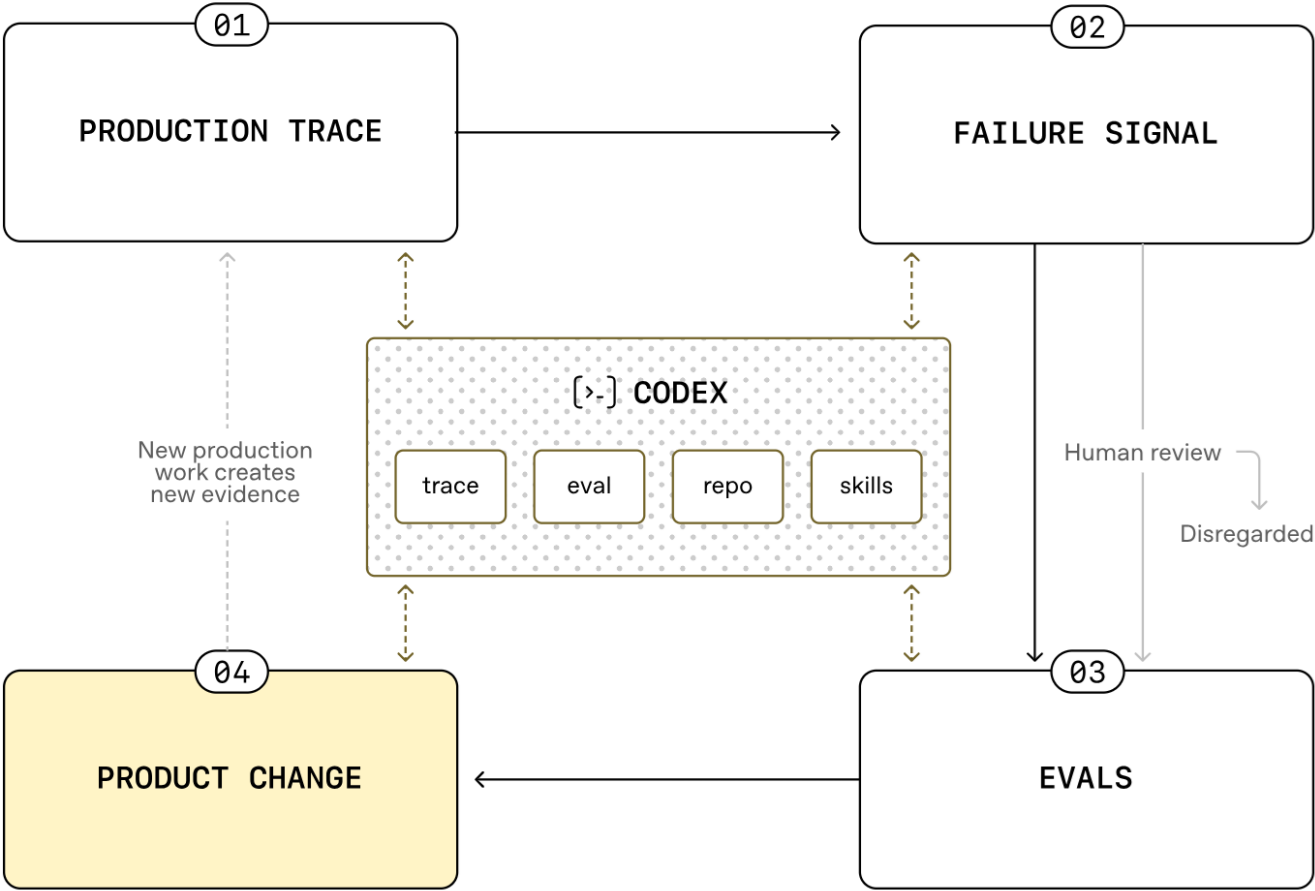
1. **Stay close to practitioners:** Those doing the work should guide product learning. Their insights identify critical errors and inform workflow priorities.
2. **Build the product so production creates evidence:** The product must capture the full workflow, from source material and extracted fields to downstream submission and expert correction, not just inputs and outputs.
3. **Create a Codex-driven improvement loop:** Visible, structured production issues become findings, evals, and engineering tasks. Codex can then investigate, propose changes, validate them with targeted and regression evals, accelerating product iteration beyond manual cycles.

Rental property example



A rental property source package is normalized into cited fields before those are mapped to downstream tax engine concepts.

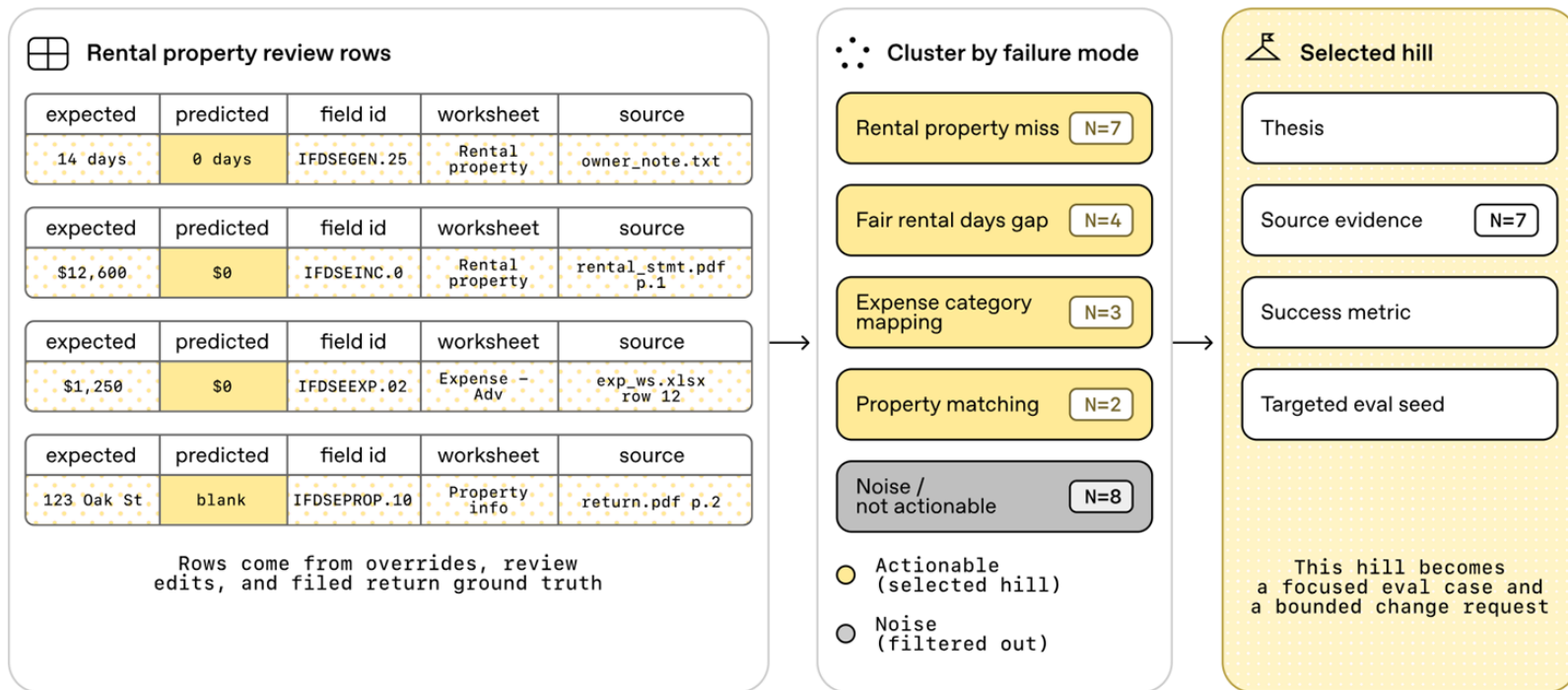
A single tax AI improvement loop





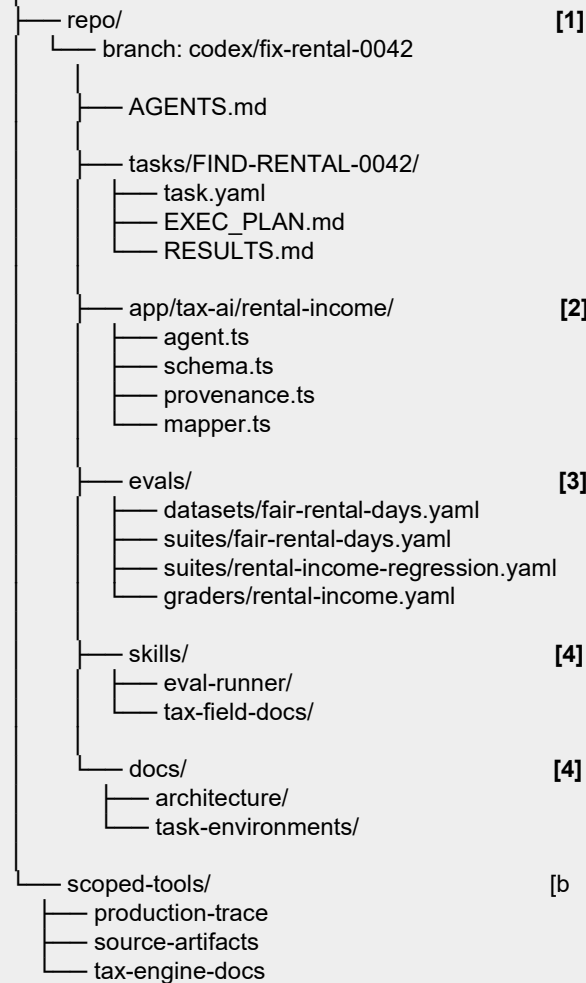
Corrections become hill candidates

Review rows are grouped into a small set of failure patterns



Rental property review rows separate recurring product failures from expected noise, then turn the actionable cases into evaluation targets that give Codex a hill to climb.

/candidates/FIND-RENTAL-0042/



The writable worktree **[1]** from read-only production context **[5]**.

The worktree contains the scoped product surface Codex can inspect or modify **[2]**, the targeted and regression evals that define success **[3]**, and reusable skills/docs that encode how to run the task and respect prior decisions **[4]**.

The read-only context provides the production trace, source documents, Tax AI prediction, finalized return, and tax-engine field documentation, so Codex can investigate the failure without mutating the underlying evidence.

Project Glasswing: what Mythos showed us

2026-05-18



Grant Bourzikas

9 min read

This post is also available in [简体中文](#), [Français](#), [Deutsch](#), [Italiano](#), [日本語](#), [한국어](#), [Português](#), [Español](#), [Nederlands](#) and [繁體中文](#).



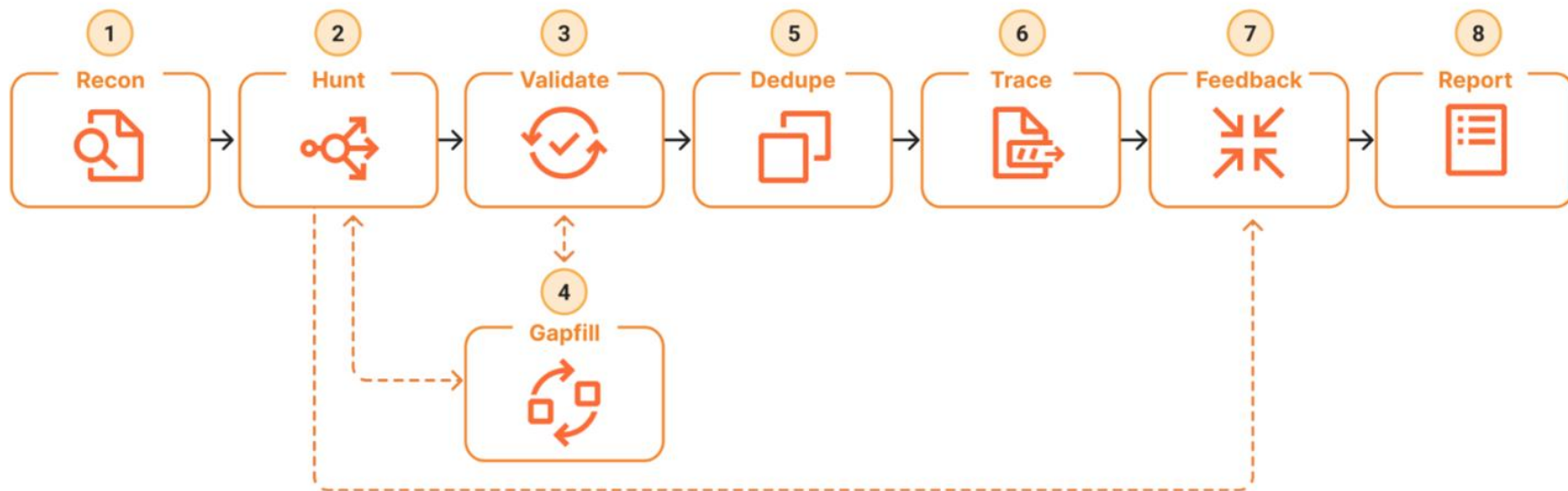
Why pointing a generic coding agent at a repo doesn't work

- **Context:** Coding agents are optimized for focused, sequential tasks (like building a feature or fixing a bug), holding one hypothesis at a time. This is ill-suited for vulnerability research, which is inherently narrow and parallel. **A single agent session on a large repository covers less than one percent of the surface before its context window fills**, potentially losing crucial earlier findings.
- **Throughput:** A single-stream agent does one task at a time, but real codebases require simultaneously testing many hypotheses against multiple components, with the ability to expand findings.

What a harness actually fixes

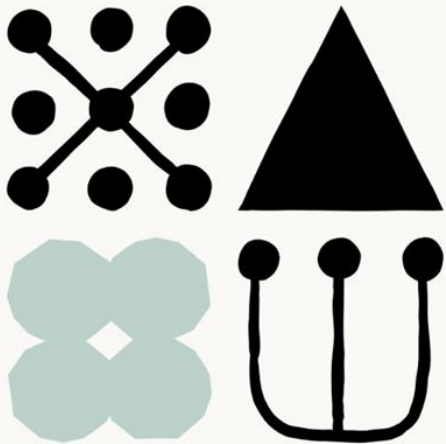
- **Narrow scope yields better findings** - Specific instructions ("Look for command injection in this function...") outperform vague ones ("Find vulnerabilities in this repository"), mimicking human researcher efficiency.
- **Adversarial review reduces noise** - Using a second, differently prompted agent to review initial findings drastically lowers false positives; deliberate disagreement is more effective than self-correction.
- **Splitting the chain improves reasoning** - Separately asking narrow questions ("Is this code buggy?" and "Is it reachable?") leads to better answers than combining them.
- **Parallel narrow tasks beat one exhaustive agent** - Utilizing many agents on tightly scoped questions and deduplicating results achieves better coverage than relying on a single comprehensive agent.

Vulnerability discovery harness



Stage	What it does	Why it matters
1 Recon	An agent reads the repository from the top down, fans out to subagents responsible for each subsystem, and produces an architecture document covering build commands, trust boundaries, entry points, and likely attack surface. It also generates the initial queue of tasks for the next stage.	Gives every downstream agent shared context. Cuts the wander problem.
2 Hunt	Each task is one attack class paired with a scope hint. Hunters (the agents that actually look for bugs) run concurrently, typically around fifty at once, each fanning out to a handful of exploration subagents. Each hunter has access to tools that compile and run proof-of-concept code in a per-task scratch directory.	This is where most of the work happens. Many narrow tasks in parallel, not one exhaustive agent.
3 Validate	An independent agent re-reads the code and tries to disprove the original finding. It uses a different prompt and has no ability to emit new findings of its own.	Catches a meaningful fraction of the noise the hunter wouldn't catch when reviewing its own work.
4 Gapfill	Hunters flag areas they touched but didn't cover thoroughly. Those areas get re-queued for another pass.	Counteracts the model's tendency to drift toward attack classes it has already had success with.

5 Dedupe	Findings that share the same root cause collapse into a single record.	Variant analysis is a feature, not a way to inflate the queue with duplicates.
6 Trace	For each confirmed finding in a shared library, a tracer agent fans out (one instance per consumer repository), uses a cross-repo symbol index, and decides whether attacker-controlled input actually reaches the bug from outside the system.	Turns "there is a flaw" into "there is a reachable vulnerability." This is the stage that matters most.
7 Feedback	Reachable traces become new hunt tasks in the consumer repositories where the bug is actually exposed.	Closes the loop. The pipeline gets better as it runs.
8 Report	An agent writes a structured report against a predefined schema, fixes any validation errors against that schema itself, and submits the report to an ingest API.	Output is queryable data, not free-form prose.



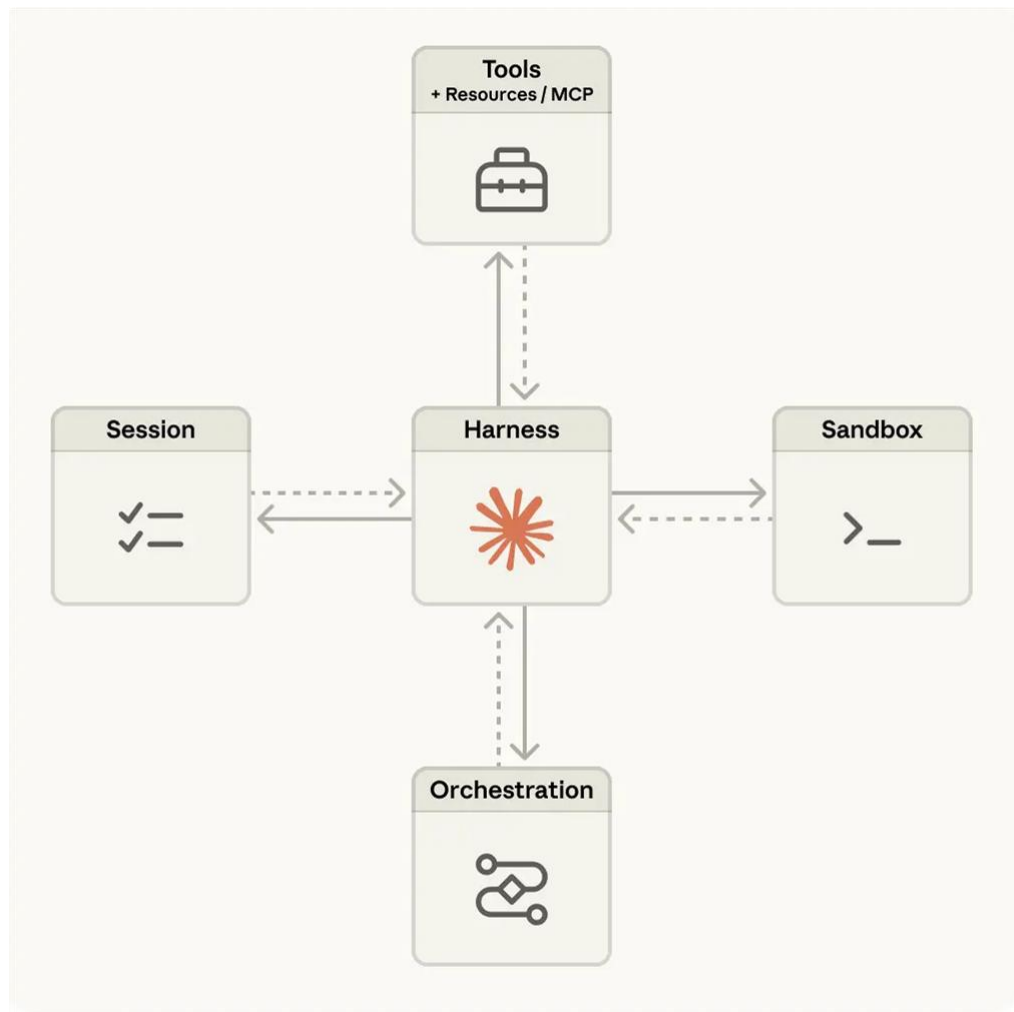
Scaling Managed Agents: Decoupling the brain from the hands

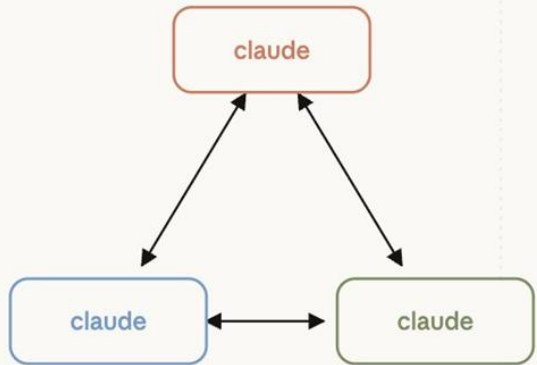
Published Apr 08, 2026

Harnesses encode assumptions that go stale as models improve. Managed Agents—our hosted service for long-horizon agent work—is built around interfaces that stay stable as harnesses change.

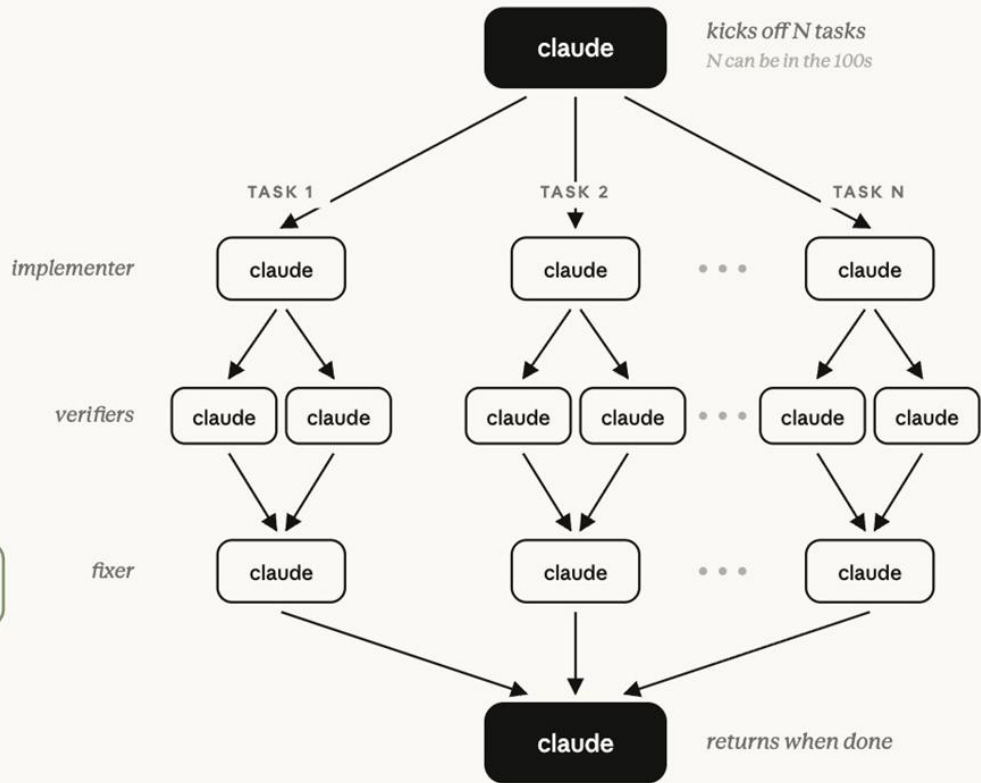
<https://www.anthropic.com/engineering/harness-design-long-running-apps>

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.





AGENT TEAMS



DYNAMIC WORKFLOWS

Harness Engineering Conclusion

- It is always good practice to **experiment with the model** you're building against, **read its traces on realistic problems**, and tune its performance to achieve your desired outcomes.
- On more complex tasks, there is sometimes headroom from decomposing the task and applying specialized agents to each aspect of the problem.
- **When a new model lands, it is generally good practice to re-examine a harness**, stripping away pieces that are no longer load-bearing to performance and adding new pieces to achieve greater capability that may not have been possible before.